SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

EXPERIMENTS

Name of the Student	**A Afran**
Reg. No	**192325024**
GitHub Link	**https://github.com/Faizze-PI/MLA0403-CO1- A3.git**

COURSE NAME: Deep Learning COURSE CODE: MLA0403 FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I A Afran / 192325024 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Name of the student: A Afran
QUESTIONS:10 Total Marks:50

CO1- AT3 Experiments
Questions

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer :

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer :

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer :

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

b) Answer :

c) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Answer :

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer :

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer :

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a)Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

b)Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer :

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

ANSWERS

Question 1: Learning Algorithms - Linear Regression using Gradient Descent

Problem Statement

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Planning

- Load the Housing Prices dataset and encode categorical (yes/no) columns as 0/1, and the furnishing status as an ordinal value.
- Separate the feature matrix X (area, bedrooms, bathrooms, etc.) and target vector y (price). • Normalize (standardize) both X and y so that gradient descent converges smoothly regardless of feature scale.
- Prepend a column of ones to X to account for the bias/intercept term.
- Initialize the parameter vector theta randomly, then define the cost function (Mean Squared Error / 2).

• Run gradient descent for a fixed number of iterations, updating theta using the gradient of the cost function and recording the cost at every step.

• Plot the recorded cost values against iteration number to visualize the learning curve. • Convert normalized predictions back to the original price scale and evaluate the model using MSE, RMSE and R-squared.

• Plot Actual vs Predicted prices to visually assess model fit.

Pseudocode

```
BEGIN
 LOAD dataset Housing.csv
 ENCODE categorical columns (yes/no -> 1/0, furnishingstatus -> 0/1/2)  X <- feature columns ; y <-
price column
 NORMALIZE X and y (zero mean, unit variance)
 X <- [1, X] // add bias column

 INITIALIZE theta randomly
 SET learning_rate, iterations

 FUNCTION compute_cost(X, y, theta):
 predictions <- X . theta
 RETURN (1/2m) * SUM((predictions - y)^2)

 FUNCTION gradient_descent(X, y, theta, lr, iters):
 FOR i in 1..iters:
 predictions <- X . theta
 errors <- predictions - y
 gradient <- (1/m) * X^T . errors
 theta <- theta - lr * gradient
 RECORD compute_cost(X, y, theta)
 RETURN theta, cost_history

                                                    theta_final, cost_history <- gradient_descent(X, y, theta, learning_rate, iterations)

 PLOT cost_history vs iterations // learning curve
 CONVERT predictions back to real price scale
 COMPUTE MSE, RMSE, R2
 PLOT actual vs predicted prices
END
```

Code (Python)

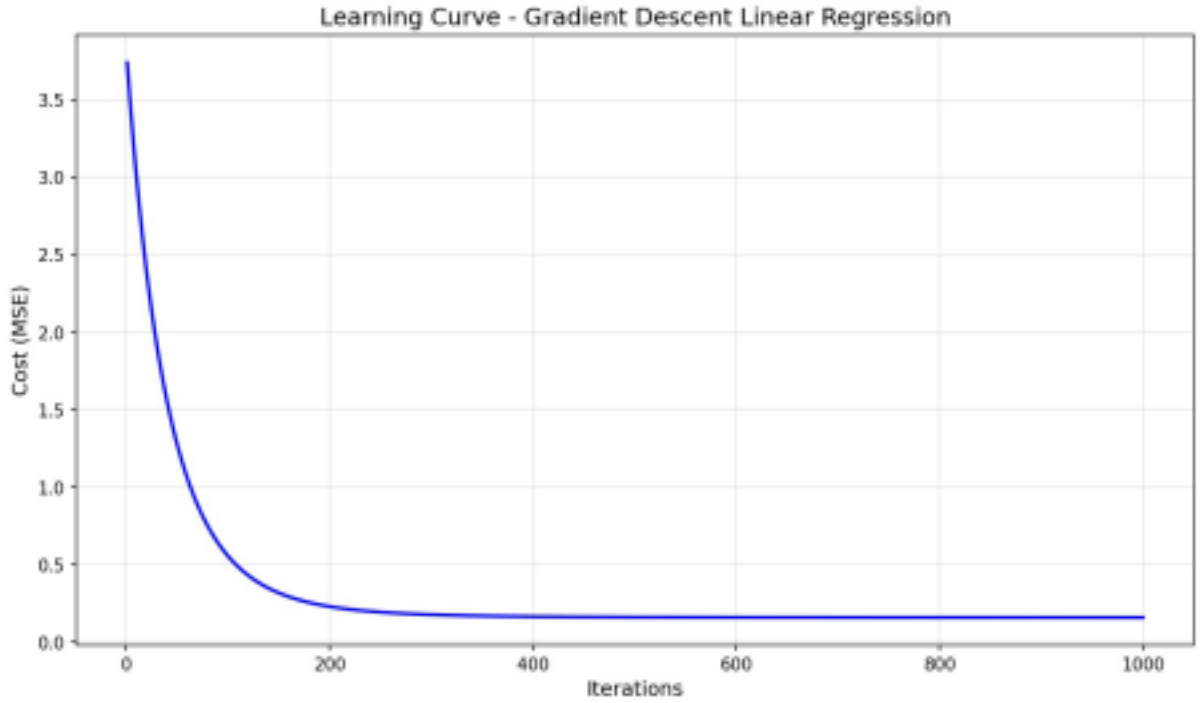
```
# ========================================================================
# QUESTION 1: Learning Algorithms
# Implement a simple learning algorithm (Linear Regression) using Gradient Descent.
# Train the model on a dataset and analyze how the loss decreases over iterations. # Plot the learning curve.
# ======================================================================== #
Dataset: Housing Prices Dataset
# Link: https://www.kaggle.com/datasets/yasserh/housing-prices-dataset
# ========================================================================

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

data = pd.read_csv('Housing.csv')

data['mainroad'] = data['mainroad'].map({'yes': 1, 'no': 0})
data['guestroom'] = data['guestroom'].map({'yes': 1, 'no': 0})
data['basement'] = data['basement'].map({'yes': 1, 'no': 0})
data['hotwaterheating'] = data['hotwaterheating'].map({'yes': 1, 'no': 0})
data['airconditioning'] = data['airconditioning'].map({'yes': 1, 'no': 0})
data['prefarea'] = data['prefarea'].map({'yes': 1, 'no': 0})
data['furnishingstatus'] = data['furnishingstatus'].map({'unfurnished': 0, 'semi-furnished': 1,  'furnished': 2})

X = data[['area', 'bedrooms', 'bathrooms', 'stories', 'mainroad', 'guestroom',  'basement', 'hotwaterheating',
'airconditioning', 'parking', 'prefarea',  'furnishingstatus']].values
y = data['price'].values

X_mean = np.mean(X, axis=0)
X_std = np.std(X, axis=0)
X_normalized = (X - X_mean) / X_std

y_mean = np.mean(y)
y_normalized = (y - y_mean) / np.std(y)
```

```
X_normalized = np.c_[np.ones(X_normalized.shape[0]), X_normalized]

def compute_cost(X, y, theta):
 m = len(y)
 predictions = X.dot(theta)
 cost = (1 / (2 * m)) * np.sum((predictions - y) ** 2)
 return cost

def gradient_descent(X, y, theta, learning_rate, iterations):
 m = len(y)
 cost_history = []

 for i in range(iterations):
 predictions = X.dot(theta)
 errors = predictions - y
 gradient = (1 / m) * X.T.dot(errors)
 theta = theta - learning_rate * gradient
 cost = compute_cost(X, y, theta)
 cost_history.append(cost)

 return theta, cost_history

np.random.seed(42)
theta_initial = np.random.randn(X_normalized.shape[1])

learning_rate = 0.01
iterations = 1000

theta_final, cost_history = gradient_descent(X_normalized, y_normalized, theta_initial, learning_rate,  iterations)

print("Final Parameters (theta):")
print(theta_final)
print(f"\nFinal Cost: {cost_history[-1]:.6f}")

plt.figure(figsize=(10, 6))
plt.plot(range(1, iterations + 1), cost_history, 'b-', linewidth=2)
plt.xlabel('Iterations', fontsize=12)
plt.ylabel('Cost (MSE)', fontsize=12)
plt.title('Learning Curve - Gradient Descent Linear Regression', fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('Q1_learning_curve.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n--- Model Performance ---")
predictions = X_normalized.dot(theta_final)
predictions_actual = predictions * np.std(y) + y_mean
mse = np.mean((predictions_actual - y) ** 2)
 rmse = np.sqrt(mse)
ss_res = np.sum((y - predictions_actual) ** 2)
ss_tot = np.sum((y - y_mean) ** 2)
 r2 = 1 - (ss_res / ss_tot)

print(f"Mean Squared Error: {mse:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")
print(f"R-squared Score: {r2:.4f}")

plt.figure(figsize=(10, 6))
plt.scatter(y, predictions_actual, alpha=0.5, color='blue', edgecolors='k', linewidth=0.5) plt.plot([y.min(), y.max()], [y.min(), y.max()],
'r--', linewidth=2, label='Perfect Prediction') plt.xlabel('Actual Price', fontsize=12)
plt.ylabel('Predicted Price', fontsize=12)
plt.title('Actual vs Predicted Housing Prices', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('Q1_actual_vs_predicted.png', dpi=150, bbox_inches='tight')
plt.show()

print("\nQ1 completed successfully!")
```

Dataset Used

Housing Prices Dataset (Kaggle: yasserh/housing-prices-dataset) — loaded from Housing.csv, containing area, bedrooms, bathrooms, stories, and other housing attributes with target variable 'price'.

Output



Learning curve: Cost (MSE) vs Iterations, showing rapid convergence within the first ~200 iterations.

Actual vs Predicted housing prices, compared against the ideal y = x line.

Question 2: Maximum Likelihood Estimation (MLE)

Problem Statement

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Planning

• Define the true parameters of a normal distribution: mean (mu) = 5.0 and variance (sigma^2) = 2.0. • Generate a synthetic sample of 1000 points from N(mu, sigma^2) using numpy's random normal generator.

• Derive/implement the MLE formulas for the mean and variance of a normal distribution: mu_hat is the sample mean, sigma2_hat is the average squared deviation from the sample mean. • Compute the MLE estimates from the generated sample and compare them against the true parameters, reporting the absolute errors.

• Plot the sample histogram together with the true and MLE-estimated normal density curves for visual comparison.

• Repeat the estimation for increasing sample sizes (10 to 5000) to show how the MLE estimation error shrinks as sample size grows (consistency of MLE).

Pseudocode

```
BEGIN
 SET true_mean = 5.0, true_variance = 2.0
 data <- SAMPLE 1000 points from Normal(true_mean, sqrt(true_variance))

 FUNCTION mle_estimates(samples):
 n <- length(samples)
 mean_hat <- SUM(samples) / n
 var_hat <- SUM((samples - mean_hat)^2) / n
 RETURN mean_hat, var_hat

 mean_hat, var_hat <- mle_estimates(data)
 PRINT true vs estimated parameters and errors

 PLOT histogram(data) overlaid with:
 true normal PDF(true_mean, true_variance)
 MLE normal PDF(mean_hat, var_hat)

 FOR n in [10, 50, 100, 500, 1000, 5000]:
 sample <- SAMPLE n points from Normal(true_mean, sqrt(true_variance))  m_hat, v_hat <-
mle_estimates(sample)
 RECORD |m_hat - true_mean|, |v_hat - true_variance|

 PLOT estimation error vs sample size (log scale)
END
```

Code (Python)

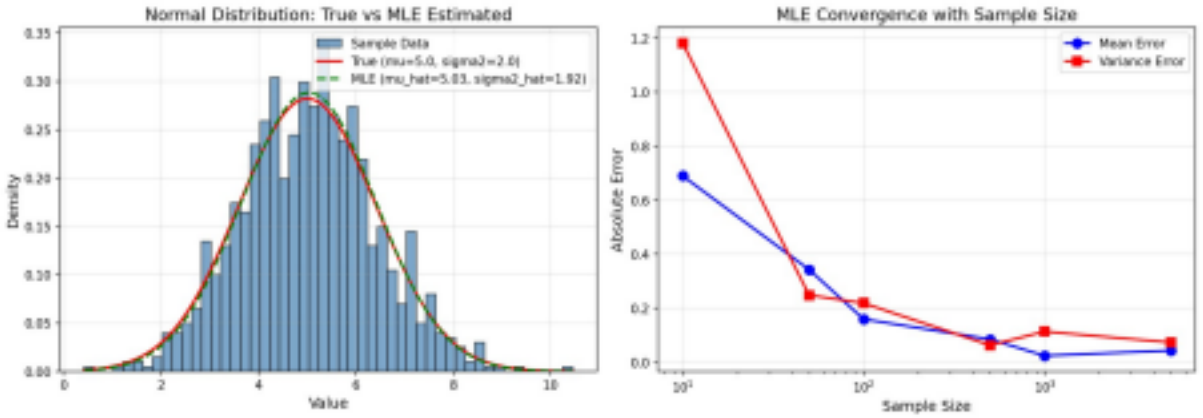
```
# ============================================================================
# QUESTION 2: Maximum Likelihood Estimation (MLE)
# Generate synthetic data from a normal distribution and use MLE to estimate
# the mean and variance. Compare estimated values with actual parameters.
# ============================================================================
# Dataset: Synthetic Data (Normal Distribution - No external dataset needed)
# Generated using numpy.random.normal()
# ============================================================================

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

np.random.seed(42)

true_mean = 5.0
true_variance = 2.0
true_std = np.sqrt(true_variance)
sample_size = 1000
data = np.random.normal(loc=true_mean, scale=true_std, size=sample_size)

def mle_estimates(samples):
 n = len(samples)
 mean_estimate = np.sum(samples) / n
 variance_estimate = np.sum((samples - mean_estimate) ** 2) / n
 return mean_estimate, variance_estimate

estimated_mean, estimated_variance = mle_estimates(data)

print("=" * 50)
print("Maximum Likelihood Estimation (MLE) Results")
print("=" * 50)
print(f"\nTrue Parameters:")
print(f" Mean (mu): {true_mean}")
print(f" Variance (sigma2): {true_variance}")
print(f" Std Dev (sigma): {true_std:.4f}")

print(f"\nMLE Estimated Parameters:")
print(f" Mean (mu_hat): {estimated_mean:.4f}")
print(f" Variance (sigma2_hat): {estimated_variance:.4f}")
print(f" Std Dev (sigma_hat): {np.sqrt(estimated_variance):.4f}")

print(f"\nEstimation Errors:")
print(f" Mean Error: {abs(estimated_mean - true_mean):.4f}")
print(f" Variance Error: {abs(estimated_variance - true_variance):.4f}")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
axes[0].hist(data, bins=50, density=True, alpha=0.7, color='steelblue', edgecolor='black', label='Sample  Data')
x = np.linspace(min(data), max(data), 1000)
true_pdf = (1 / (true_std * np.sqrt(2 * np.pi))) * np.exp(-0.5 * ((x - true_mean) / true_std) ** 2) estimated_pdf = (1 /
(np.sqrt(estimated_variance) * np.sqrt(2 * np.pi))) * np.exp(-0.5 * ((x -  estimated_mean) / np.sqrt(estimated_variance)) ** 2)

axes[0].plot(x, true_pdf, 'r-', linewidth=2, label=f'True (mu={true_mean}, sigma2={true_variance})') axes[0].plot(x, estimated_pdf, 'g--',
linewidth=2, label=f'MLE (mu_hat={estimated_mean:.2f},  sigma2_hat={estimated_variance:.2f})')
axes[0].set_xlabel('Value', fontsize=12)
axes[0].set_ylabel('Density', fontsize=12)
axes[0].set_title('Normal Distribution: True vs MLE Estimated', fontsize=14)
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.3)

sample_sizes = [10, 50, 100, 500, 1000, 5000]
mean_errors = []
var_errors = []

for n in sample_sizes:
 samples = np.random.normal(true_mean, true_std, n)
 m_mean, m_var = mle_estimates(samples)
 mean_errors.append(abs(m_mean - true_mean))
 var_errors.append(abs(m_var - true_variance))

axes[1].plot(sample_sizes, mean_errors, 'bo-', linewidth=2, markersize=8, label='Mean Error') axes[1].plot(sample_sizes,
var_errors, 'rs-', linewidth=2, markersize=8, label='Variance Error') axes[1].set_xlabel('Sample Size', fontsize=12)
axes[1].set_ylabel('Absolute Error', fontsize=12)
axes[1].set_title('MLE Convergence with Sample Size', fontsize=14)
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)
axes[1].set_xscale('log')

plt.tight_layout()
plt.savefig('Q2_MLE_results.png', dpi=150, bbox_inches='tight')
plt.close()

print("\n" + "=" * 50)
print("Q2 completed successfully!")
print("Plot saved as Q2_MLE_results.png")
```

Dataset Used

Synthetic data generated using numpy.random.normal() — no external dataset required. 1000 samples drawn from a Normal(mu=5.0, sigma^2=2.0) distribution.

Output



Left: sample histogram with true vs MLE-fitted normal curves (mu_hat=5.03, sigma2_hat=1.92). Right: MLE estimation error shrinking as sample size increases, confirming consistency of the estimator.

Question 3: Building Machine Learning Algorithms - Classification Pipeline

Problem Statement

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Planning

- Load the Breast Cancer Wisconsin (Diagnostic) dataset and drop non-informative columns (id, unnamed columns).
- Encode the target 'diagnosis' column (Malignant/Benign) into numeric labels using LabelEncoder. • Split the data into training (80%) and test (20%) sets using stratified

sampling to preserve class balance.

• Standardize features using StandardScaler (fit on training data, applied to both train and test). • Train three different classifiers: Logistic Regression, Support Vector Machine (RBF kernel), and Random Forest.

• Evaluate each model on the test set using accuracy score and confusion matrix.

• Identify the best-performing model and print a detailed classification report (precision, recall, F1- score).

• Visualize all three confusion matrices as heatmaps and compare model accuracies with a bar chart.

Pseudocode

```
BEGIN
LOAD data.csv (Breast Cancer Wisconsin dataset)
DROP id, Unnamed: 32 columns
ENCODE diagnosis column -> numeric labels (0/1)

X <- feature columns ; y <- diagnosis labels
SPLIT X, y INTO X_train, X_test, y_train, y_test (80/20, stratified)  SCALE: fit StandardScaler on

X_train, transform X_train and X_test

MODELS <- {LogisticRegression, SVM(rbf), RandomForest}
FOR each model in MODELS:
model.fit(X_train_scaled, y_train)
y_pred <- model.predict(X_test_scaled)
accuracy <- accuracy_score(y_test, y_pred)
cm <- confusion_matrix(y_test, y_pred)
STORE accuracy, cm

best_model <- model with highest accuracy
PRINT classification_report(best_model)

PLOT confusion matrix heatmaps for all 3 models
PLOT bar chart comparing model accuracies
END
```

Code (Python)

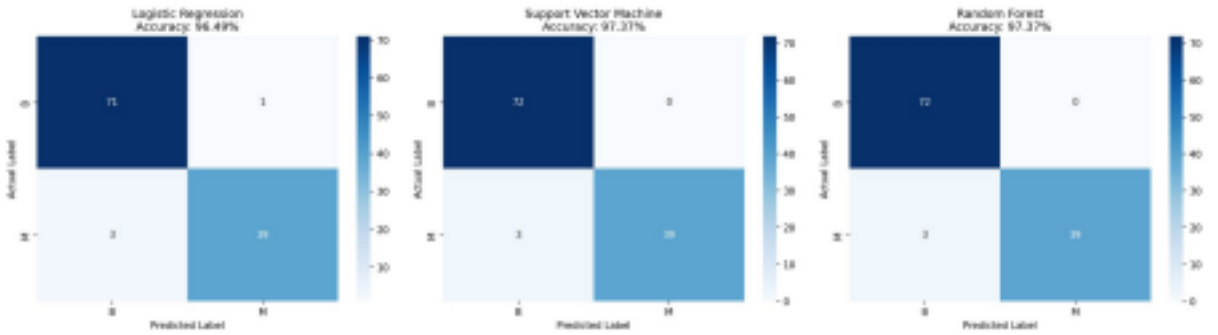
```
# ============================================================================
# QUESTION 3: Building Machine Learning Algorithms
# Build a complete machine learning model (data preprocessing, training, testing)
# using any classification dataset and evaluate performance using accuracy and
# confusion matrix.
# ============================================================================
# Dataset: Breast Cancer Wisconsin (Diagnostic) Data Set
# Link: https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data
# ============================================================================

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report import seaborn as sns

data = pd.read_csv('data.csv')

print("Dataset Shape:", data.shape)
print("\nFirst 5 rows:")
print(data.head())

data = data.drop('id', axis=1, errors='ignore')
data = data.drop('Unnamed: 32', axis=1, errors='ignore')

le = LabelEncoder()
data['diagnosis'] = le.fit_transform(data['diagnosis'])

X = data.drop('diagnosis', axis=1).values
y = data['diagnosis'].values

print(f"\nFeatures shape: {X.shape}")
print(f"Target distribution: {np.bincount(y)}")
```

```
print(f"Classes: {le.classes_}")

                                                X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"\nTraining set size: {X_train_scaled.shape[0]}")
print(f"Test set size: {X_test_scaled.shape[0]}")

models = {
 'Logistic Regression': LogisticRegression(max_iter=10000, random_state=42),  'Support Vector Machine':
SVC(kernel='rbf', random_state=42),
 'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42) }

results = {}

for name, model in models.items():
 model.fit(X_train_scaled, y_train)
 y_pred = model.predict(X_test_scaled)
 accuracy = accuracy_score(y_test, y_pred)
 cm = confusion_matrix(y_test, y_pred)
 results[name] = {'accuracy': accuracy, 'confusion_matrix': cm, 'predictions': y_pred}  print(f"\n{name}:")
 print(f" Accuracy: {accuracy * 100:.2f}%")

best_model_name = max(results, key=lambda x: results[x]['accuracy'])
best_result = results[best_model_name]

print(f"\n{'='*60}")
print(f"Best Model: {best_model_name}")
print(f"Accuracy: {best_result['accuracy'] * 100:.2f}%")
print(f"{'='*60}")

print(f"\nClassification Report ({best_model_name}):")
print(classification_report(y_test, best_result['predictions'], target_names=le.classes_)) fig, axes = plt.subplots(1, 3,

figsize=(18, 5))

for idx, (name, result) in enumerate(results.items()):
 cm = result['confusion_matrix']
 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[idx],
 xticklabels=le.classes_, yticklabels=le.classes_)
 axes[idx].set_title(f'{name}\nAccuracy: {result["accuracy"]*100:.2f}%', fontsize=12)  axes[idx].set_ylabel('Actual Label',
fontsize=10)
 axes[idx].set_xlabel('Predicted Label', fontsize=10)

plt.tight_layout()
plt.savefig('Q3_confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.close()

model_names = list(results.keys())
accuracies = [results[name]['accuracy'] * 100 for name in model_names]

plt.figure(figsize=(8, 5))
bars = plt.bar(model_names, accuracies, color=['steelblue', 'coral', 'forestgreen'], edgecolor='black') plt.ylabel('Accuracy (%)', fontsize=12)
plt.title('Model Comparison - Breast Cancer Classification', fontsize=14)
plt.ylim(90, 100)
plt.grid(axis='y', alpha=0.3)
for bar, acc in zip(bars, accuracies):
 plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.2,
 f'{acc:.2f}%', ha='center', va='bottom', fontsize=11, fontweight='bold')

plt.tight_layout()
plt.savefig('Q3_model_comparison.png', dpi=150, bbox_inches='tight')
plt.close()

print("\nQ3 completed successfully!")
print("Plots saved as Q3_confusion_matrices.png and Q3_model_comparison.png")
```

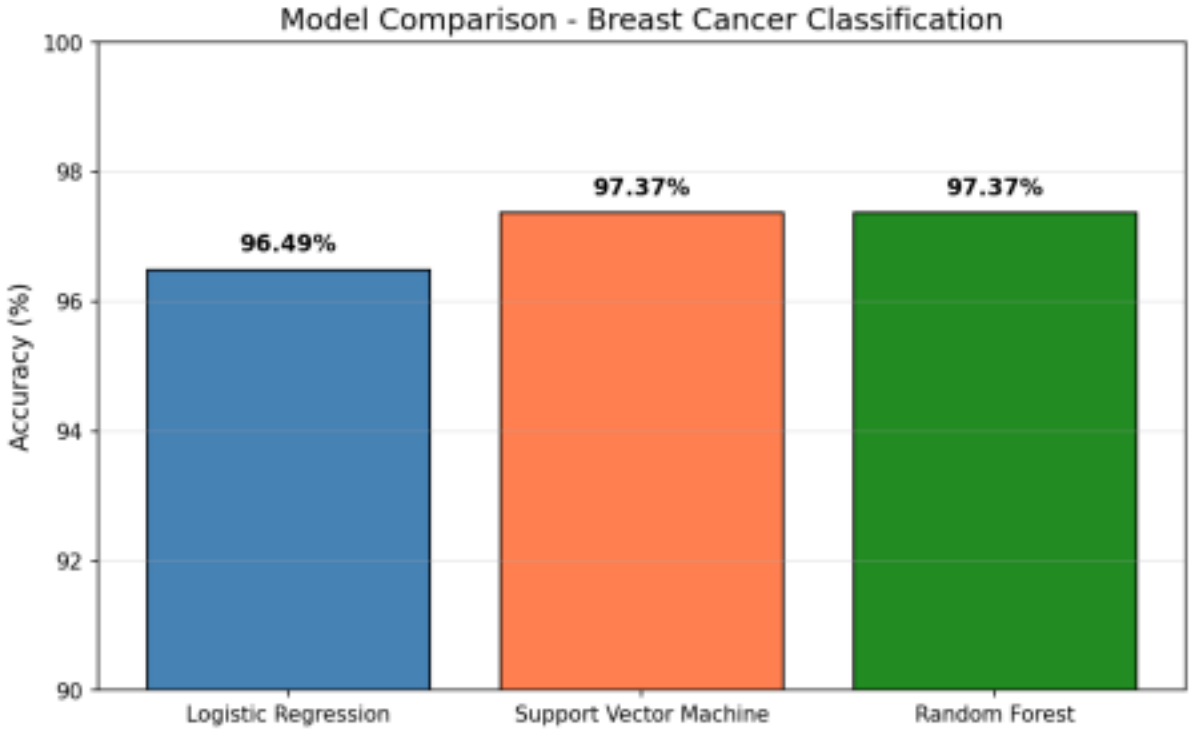
Dataset Used

Breast Cancer Wisconsin (Diagnostic) Data Set (Kaggle: uciml/breast-cancer-wisconsin-data) — loaded from data.csv, with 30 numeric features per cell nucleus and a binary diagnosis label (Malignant / Benign).

Output



Confusion matrices for Logistic Regression (96.49%), SVM (97.37%), and Random Forest (97.37%) on the breast cancer test set.



Bar chart comparing test accuracy across the three classification models.

Question 4: Neural Networks - One Hidden Layer on Non-Linear Data

Problem Statement

Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Planning

• Generate a non-linearly separable 'moons' dataset (1000 samples) using sklearn's make_moons, and split into train/test sets.

• Standardize the input features so the network trains stably.

• Design a neural network with: an input layer (2 neurons), one hidden layer (8 neurons, ReLU activation), and an output layer (1 neuron, Sigmoid activation for binary classification). • Implement forward propagation (matrix multiplications + activations) and the binary cross-entropy loss function.

• Implement backpropagation to compute gradients of the loss with respect to all weights and biases. • Train the network for 1000 epochs using gradient descent, recording the loss at each epoch. • Evaluate final training and test accuracy, and plot: the loss curve, the learned non-linear decision boundary, and an accuracy bar chart.

Pseudocode

```
BEGIN
 X, y <- GENERATE make_moons(1000 samples, noise=0.2)
```

```
SPLIT into X_train, X_test, y_train, y_test
SCALE features using StandardScaler

INITIALIZE W1, b1 (input->hidden), W2, b2 (hidden->output) randomly

FUNCTION forward(X):
z1 <- X.W1 + b1 ; a1 <- ReLU(z1)
z2 <- a1.W2 + b2 ; a2 <- Sigmoid(z2)
RETURN a2

FUNCTION backward(X, y_true, y_pred):
dz2 <- y_pred - y_true
dW2 <- (1/m) * a1^T . dz2 ; db2 <- mean(dz2)
da1 <- dz2 . W2^T
dz1 <- da1 * ReLU'(z1)
dW1 <- (1/m) * X^T . dz1 ; db1 <- mean(dz1)
RETURN dW1, db1, dW2, db2

FOR epoch in 1..1000:
y_pred <- forward(X_train)
loss <- binary_cross_entropy(y_train, y_pred)
gradients <- backward(X_train, y_train, y_pred)
UPDATE W1, b1, W2, b2 using gradients and learning_rate
RECORD loss

COMPUTE train_accuracy, test_accuracy
PLOT loss curve, decision boundary (contour of forward() over grid), accuracy bar  chart
END
```

Code (Python)

```
# ============================================================================
# QUESTION 4: Neural Networks
# Design a simple neural network with one hidden layer and train it on a small
# dataset. Analyze how the network learns non-linear patterns.
# ============================================================================
# Dataset: Moons Dataset (Non-Linear Classification)
# Link: https://www.kaggle.com/datasets/emadmakhlouf/linearly-inseperable-dataset
# Note: Using sklearn.datasets.make_moons() for synthetic generation
# ============================================================================

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)
y = y.reshape(-1, 1)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

def sigmoid(z):
return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def sigmoid_derivative(a):
return a * (1 - a)

def relu(z):
return np.maximum(0, z)

def relu_derivative(z):
return (z > 0).astype(float)

class NeuralNetwork:
def __init__(self, input_size, hidden_size, output_size):
self.W1 = np.random.randn(input_size, hidden_size) * 0.5
self.b1 = np.zeros((1, hidden_size))
self.W2 = np.random.randn(hidden_size, output_size) * 0.5
self.b2 = np.zeros((1, output_size))

def forward(self, X):
self.z1 = X.dot(self.W1) + self.b1
self.a1 = relu(self.z1)
self.z2 = self.a1.dot(self.W2) + self.b2
self.a2 = sigmoid(self.z2)
```

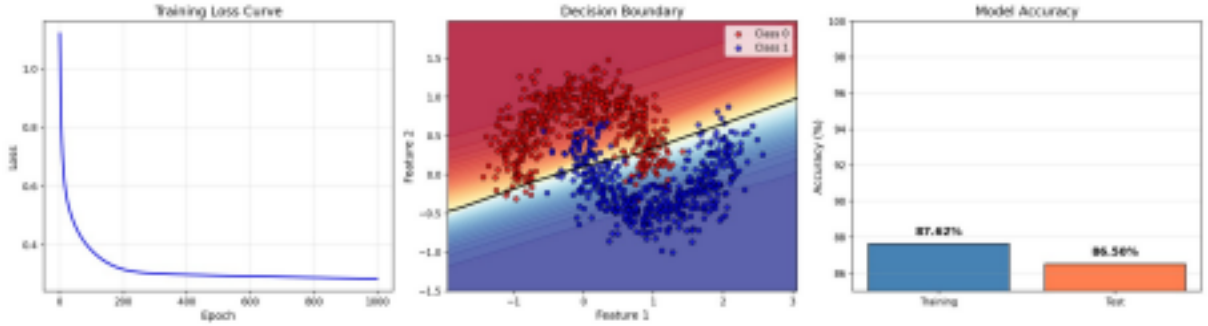
```
return self.a2

def compute_loss(self, y_true, y_pred):
m = y_true.shape[0]
loss = -np.mean(y_true * np.log(y_pred + 1e-8) + (1 - y_true) * np.log(1 - y_pred + 1e-8))  return loss

def backward(self, X, y_true, y_pred):
m = X.shape[0]

dz2 = y_pred - y_true
dW2 = (1 / m) * self.a1.T.dot(dz2)
db2 = (1 / m) * np.sum(dz2, axis=0, keepdims=True)

da1 = dz2.dot(self.W2.T)
dz1 = da1 * relu_derivative(self.z1)
dW1 = (1 / m) * X.T.dot(dz1)
db1 = (1 / m) * np.sum(dz1, axis=0, keepdims=True)

return dW1, db1, dW2, db2

def update_weights(self, dW1, db1, dW2, db2, learning_rate):
self.W1 -= learning_rate * dW1
self.b1 -= learning_rate * db1
self.W2 -= learning_rate * dW2
self.b2 -= learning_rate * db2

def train(self, X, y, epochs, learning_rate):
losses = []
for epoch in range(epochs):
y_pred = self.forward(X)
loss = self.compute_loss(y, y_pred)
losses.append(loss)

dW1, db1, dW2, db2 = self.backward(X, y, y_pred)
self.update_weights(dW1, db1, dW2, db2, learning_rate)

if (epoch + 1) % 100 == 0:
print(f"Epoch {epoch + 1}/{epochs}, Loss: {loss:.4f}")

return losses

nn = NeuralNetwork(input_size=2, hidden_size=8, output_size=1)
print("Training Neural Network...")
print("="*50)
losses = nn.train(X_train_scaled, y_train, epochs=1000, learning_rate=0.1)

y_train_pred = nn.forward(X_train_scaled)
y_test_pred = nn.forward(X_test_scaled)

train_accuracy = np.mean((y_train_pred > 0.5).astype(int) == y_train) * 100
test_accuracy = np.mean((y_test_pred > 0.5).astype(int) == y_test) * 100

print(f"\nTraining Accuracy: {train_accuracy:.2f}%")
print(f"Test Accuracy: {test_accuracy:.2f}%")

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].plot(losses, 'b-', linewidth=2)
axes[0].set_xlabel('Epoch', fontsize=12)
axes[0].set_ylabel('Loss', fontsize=12)
axes[0].set_title('Training Loss Curve', fontsize=14)
axes[0].grid(True, alpha=0.3)

xx, yy = np.meshgrid(np.linspace(X[:, 0].min()-0.5, X[:, 0].max()+0.5, 200),
 np.linspace(X[:, 1].min()-0.5, X[:, 1].max()+0.5, 200))
grid_points = np.c_[xx.ravel(), yy.ravel()]
grid_scaled = scaler.transform(grid_points)
Z = nn.forward(grid_scaled)
Z = Z.reshape(xx.shape)

axes[1].contourf(xx, yy, Z, levels=50, cmap='RdYlBu', alpha=0.8)
axes[1].contour(xx, yy, Z, levels=[0.5], colors='black', linewidths=2)
axes[1].scatter(X[y.ravel()==0, 0], X[y.ravel()==0, 1], c='red', label='Class 0', edgecolors='k', s=30,  alpha=0.7)
axes[1].scatter(X[y.ravel()==1, 0], X[y.ravel()==1, 1], c='blue', label='Class 1', edgecolors='k', s=30,  alpha=0.7)
axes[1].set_xlabel('Feature 1', fontsize=12)
axes[1].set_ylabel('Feature 2', fontsize=12)
axes[1].set_title('Decision Boundary', fontsize=14)
axes[1].legend(fontsize=10)

axes[2].bar(['Training', 'Test'], [train_accuracy, test_accuracy], color=['steelblue', 'coral'],  edgecolor='black')
axes[2].set_ylabel('Accuracy (%)', fontsize=12)
axes[2].set_title('Model Accuracy', fontsize=14)
axes[2].set_ylim(85, 100)
axes[2].grid(axis='y', alpha=0.3)
for i, v in enumerate([train_accuracy, test_accuracy]):
 axes[2].text(i, v + 0.5, f'{v:.2f}%', ha='center', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.savefig('Q4_neural_network.png', dpi=150, bbox_inches='tight')
plt.close()

print("\nQ4 completed successfully!")
```

```
print("Plot saved as Q4_neural_network.png")
```

Dataset Used

Synthetically generated non-linear 'moons' dataset via sklearn.datasets.make_moons() (1000 samples, noise=0.2) — no external file needed.

Output



Left: training loss curve dropping from ~1.1 to ~0.28. Middle: learned non-linear decision boundary separating the two moon-shaped classes. Right: training (87.62%) vs test (86.50%) accuracy.

Question 5: Artificial Neurons - Activation Function Comparison

Problem Statement

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Planning

- Define a fixed weight vector and bias for a single artificial neuron (2 inputs).
- Define a set of 8 sample 2-D input vectors covering positive, negative, and zero-valued inputs. • Implement the neuron's linear combination z = w.x + b, computed once and reused for every activation function.
- Implement Sigmoid, ReLU, Tanh, and Leaky ReLU activation functions.
- Apply every activation function to the same z values and tabulate/print the outputs side-by-side for comparison.
- Plot each activation function's curve over a continuous input range, and bar-chart the actual outputs produced for the 8 sample inputs under each activation.

Pseudocode

```
BEGIN
 DEFINE weights = [0.6, -0.4], bias = 0.1
 DEFINE 8 sample input vectors (inputs)

 z <- inputs . weights + bias // shared linear combination

 FUNCTION sigmoid(z): RETURN 1 / (1 + e^-z)
 FUNCTION relu(z): RETURN max(0, z)
 FUNCTION tanh(z): RETURN tanh(z)
 FUNCTION leaky_relu(z): RETURN z if z>0 else alpha*z

 FOR each activation function f in [sigmoid, relu, tanh, leaky_relu]:  output <- f(z)
 PRINT output for all 8 samples

 PLOT each activation function curve over continuous z range [-5, 5]  PLOT bar charts of neuron
outputs per sample for each activation
 END
```

Code (Python)

```
# ================================================================================
# QUESTION 5: Artificial Neurons
# Implement a single artificial neuron using different activation functions
# (Sigmoid, ReLU) and compare outputs for the same input values.
# ================================================================================
```

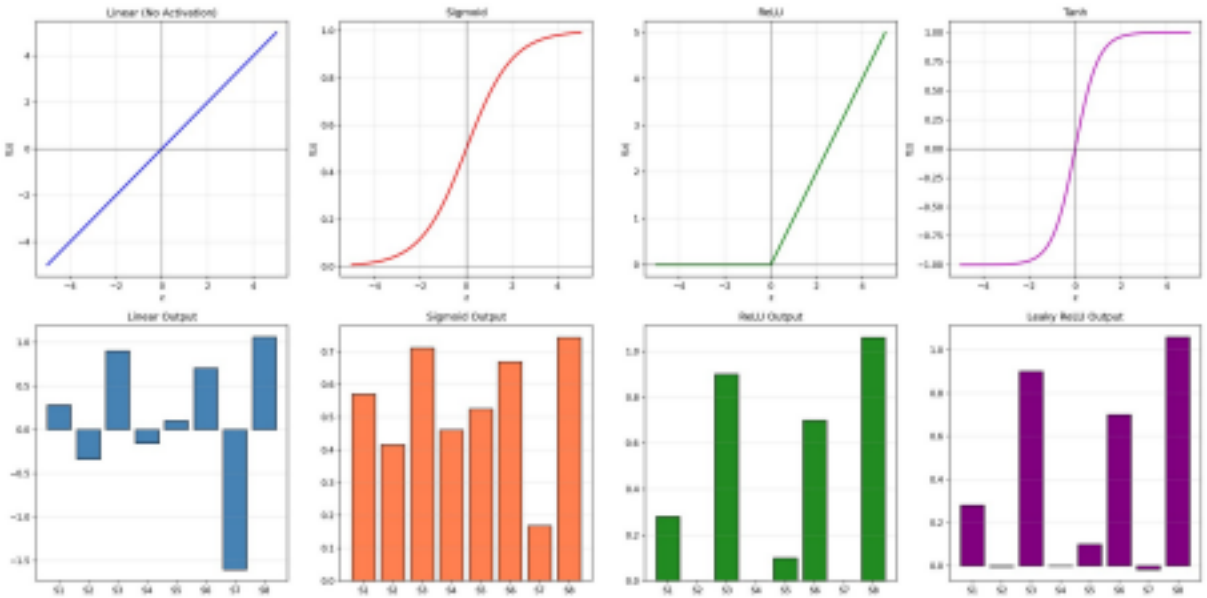
```
# Dataset: Manual Input Values (No external dataset needed)
# Using predefined input vectors for demonstration
# ================================================================================

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

def sigmoid(z):
 return 1 / (1 + np.exp(-z))

def relu(z):
 return np.maximum(0, z)

def tanh(z):
 return np.tanh(z)

def leaky_relu(z, alpha=0.01):
 return np.where(z > 0, z, alpha * z)

inputs = np.array([
[0.5, 0.3],
[-0.2, 0.8],
[1.0, -0.5],
[-0.7, -0.4],
[0.0, 0.0],
[2.0, 1.5],
[-1.5, 2.0],
[0.8, -1.2]
])

weights = np.array([0.6, -0.4])
bias = 0.1

def artificial_neuron(inputs, weights, bias, activation_func):
 z = np.dot(inputs, weights) + bias
 output = activation_func(z)
 return z, output

z_linear = inputs.dot(weights) + bias

z_sigmoid, output_sigmoid = artificial_neuron(inputs, weights, bias, sigmoid)
z_relu, output_relu = artificial_neuron(inputs, weights, bias, relu)
z_tanh, output_tanh = artificial_neuron(inputs, weights, bias, tanh)
z_leaky_relu, output_leaky_relu = artificial_neuron(inputs, weights, bias, leaky_relu)

print("="*70)
print("Artificial Neuron Implementation with Different Activation Functions")
print("="*70)
print(f"\nWeights: {weights}")
print(f"Bias: {bias}")
print(f"\nInput Samples:")
for i, inp in enumerate(inputs):
 print(f" Sample {i+1}: {inp}")

print(f"\n{'='*70}")
print("Results Comparison")
print("="*70)
print(f"\n{'Input':<20} {'Linear':<12} {'Sigmoid':<12} {'ReLU':<12} {'Tanh':<12} {'LeakyReLU':<12}") print("-"*80)
for i in range(len(inputs)):
 print(f"{str(inputs[i]):<20} {z_linear[i]:<12.4f} {output_sigmoid[i]:<12.4f} {output_relu[i]:<12.4f}  {output_tanh[i]:<12.4f}
{output_leaky_relu[i]:<12.4f}")

print(f"\n{'='*70}")
print("Activation Function Characteristics")
print("="*70)
print("\n1. Sigmoid: Output range (0, 1) - Good for probability outputs")
print("2. ReLU: Output range [0, inf) - Most popular, avoids vanishing gradient") print("3. Tanh: Output range
(-1, 1) - Zero-centered output")
print("4. Leaky ReLU: Output range (-inf, inf) - Addresses dying ReLU problem") fig, axes = plt.subplots(2,

4, figsize=(20, 10))

z_range = np.linspace(-5, 5, 1000)

axes[0, 0].plot(z_range, z_range, 'b-', linewidth=2)
axes[0, 0].set_title('Linear (No Activation)', fontsize=12)
axes[0, 0].set_xlabel('z')
axes[0, 0].set_ylabel('f(z)')
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].axhline(y=0, color='k', linewidth=0.5)
axes[0, 0].axvline(x=0, color='k', linewidth=0.5)

axes[0, 1].plot(z_range, sigmoid(z_range), 'r-', linewidth=2)
axes[0, 1].set_title('Sigmoid', fontsize=12)
axes[0, 1].set_xlabel('z')
axes[0, 1].set_ylabel('f(z)')
axes[0, 1].grid(True, alpha=0.3)
axes[0, 1].axhline(y=0, color='k', linewidth=0.5)
axes[0, 1].axvline(x=0, color='k', linewidth=0.5)
```

```
axes[0, 2].plot(z_range, relu(z_range), 'g-', linewidth=2)
axes[0, 2].set_title('ReLU', fontsize=12)
axes[0, 2].set_xlabel('z')
axes[0, 2].set_ylabel('f(z)')
axes[0, 2].grid(True, alpha=0.3)
axes[0, 2].axhline(y=0, color='k', linewidth=0.5)
axes[0, 2].axvline(x=0, color='k', linewidth=0.5)

axes[0, 3].plot(z_range, tanh(z_range), 'm-', linewidth=2)
axes[0, 3].set_title('Tanh', fontsize=12)
axes[0, 3].set_xlabel('z')
axes[0, 3].set_ylabel('f(z)')
axes[0, 3].grid(True, alpha=0.3)
axes[0, 3].axhline(y=0, color='k', linewidth=0.5)
axes[0, 3].axvline(x=0, color='k', linewidth=0.5)

x_labels = [f'S{i+1}' for i in range(len(inputs))]
x_pos = np.arange(len(inputs))

axes[1, 0].bar(x_pos, z_linear, color='steelblue', edgecolor='black')
axes[1, 0].set_title('Linear Output', fontsize=12)
axes[1, 0].set_xticks(x_pos)
axes[1, 0].set_xticklabels(x_labels)
axes[1, 0].grid(axis='y', alpha=0.3)

axes[1, 1].bar(x_pos, output_sigmoid, color='coral', edgecolor='black')
axes[1, 1].set_title('Sigmoid Output', fontsize=12)
axes[1, 1].set_xticks(x_pos)
axes[1, 1].set_xticklabels(x_labels)
axes[1, 1].grid(axis='y', alpha=0.3)

axes[1, 2].bar(x_pos, output_relu, color='forestgreen', edgecolor='black')
axes[1, 2].set_title('ReLU Output', fontsize=12)
axes[1, 2].set_xticks(x_pos)
axes[1, 2].set_xticklabels(x_labels)
axes[1, 2].grid(axis='y', alpha=0.3)

axes[1, 3].bar(x_pos, output_leaky_relu, color='purple', edgecolor='black')
axes[1, 3].set_title('Leaky ReLU Output', fontsize=12)
axes[1, 3].set_xticks(x_pos)
axes[1, 3].set_xticklabels(x_labels)
axes[1, 3].grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig('Q5_artificial_neurons.png', dpi=150, bbox_inches='tight')
plt.close()

print("\nQ5 completed successfully!")
print("Plot saved as Q5_artificial_neurons.png")
```

Dataset Used

Manually defined input vectors and weights (no external dataset needed) — 8 fixed 2-D sample points used to compare activation function behaviour.

Output



Top row: activation function curves (Linear, Sigmoid, ReLU, Tanh). Bottom row: neuron outputs for the 8 sample inputs under each activation, showing how Sigmoid squashes to (0,1), ReLU clips negatives to 0, and Leaky ReLU retains a small negative signal.

Question 6: Perceptron and Multilayer Perceptron (MLP)

Problem Statement

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non linearly separable datasets. Analyze the results.

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Planning

- Generate two datasets: a linearly separable dataset (make_classification) and a non-linearly separable dataset (make_moons).
- Implement the Perceptron algorithm from scratch: initialize weights/bias to zero, and for each sample update weights using the perceptron learning rule w <- w + lr*(y - y_pred)*x. • Train the Perceptron separately on the linear and non-linear datasets, tracking the number of misclassifications per epoch.
- Implement a simple MLP (one hidden layer, sigmoid activations) trained with backpropagation and binary cross-entropy loss.
- Train the MLP on both datasets, using the same evaluation procedure (accuracy_score) for a fair comparison.
- Compare accuracies of Perceptron vs MLP on both datasets in a summary table. • Visualize the datasets, the Perceptron's error curve, and the decision boundaries produced by the Perceptron and the MLP.

Pseudocode

```
BEGIN
X_linear, y_linear <- GENERATE make_classification (linearly separable) X_nonlinear, y_nonlinear <-
GENERATE make_moons (non-linearly separable)

CLASS Perceptron:
FUNCTION fit(X, y):
weights <- 0 ; bias <- 0
FOR each epoch:
FOR each sample (x_i, y_i):
y_pred <- step(w.x_i + b)
update <- lr * (y_i - y_pred)
weights <- weights + update * x_i
bias <- bias + update
RECORD number of misclassifications

CLASS MLP:
FUNCTION fit(X, y):
INITIALIZE W1, b1, W2, b2
FOR each epoch:
forward pass -> a2 (sigmoid output)
loss <- binary_cross_entropy(y, a2)
backward pass -> gradients
UPDATE weights via gradient descent

TRAIN Perceptron on (X_linear, y_linear) and (X_nonlinear, y_nonlinear) TRAIN MLP on (X_linear,
y_linear) and (X_nonlinear, y_nonlinear)

COMPARE accuracy_score for Perceptron vs MLP on both datasets
PLOT datasets, perceptron error curve, decision boundaries
END
```

Code (Python)

```
# ============================================================================
# QUESTION 6: Perceptron and Multilayer Perceptron (MLP)
# a) Implement the Perceptron algorithm for binary classification and test it
# on linearly separable and non-linearly separable datasets. Analyze the results. # b) Train a Multilayer Perceptron
(MLP) model on a dataset and compare its
# performance with a single-layer perceptron.
# ============================================================================ #
```

```
Dataset: Moons Dataset (Non-Linear Classification)
# Link: https://www.kaggle.com/datasets/berkayalan/sklearn-moons-data-set # Note: Using
sklearn.datasets.make_classification and make_moons for generation #
================================================================================

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

np.random.seed(42)

class Perceptron:
def __init__(self, learning_rate=0.01, n_iterations=1000):
self.learning_rate = learning_rate
self.n_iterations = n_iterations
self.weights = None
self.bias = None
self.errors = []

def fit(self, X, y):
n_samples, n_features = X.shape
self.weights = np.zeros(n_features)
self.bias = 0

for _ in range(self.n_iterations):
errors = 0
for idx, x_i in enumerate(X):
prediction = self.predict(x_i)
update = self.learning_rate * (y[idx] - prediction)  self.weights += update * x_i
self.bias += update
errors += int(update != 0.0)
self.errors.append(errors)

def predict(self, X):
linear_output = np.dot(X, self.weights) + self.bias
return np.where(linear_output >= 0, 1, 0)

class MLP:
def __init__(self, hidden_size=10, learning_rate=0.01, n_iterations=1000):  self.hidden_size =
hidden_size
self.learning_rate = learning_rate
self.n_iterations = n_iterations

def sigmoid(self, z):
return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def sigmoid_derivative(self, a):
return a * (1 - a)

def fit(self, X, y):
n_samples, n_features = X.shape
y = y.reshape(-1, 1)

self.W1 = np.random.randn(n_features, self.hidden_size) * 0.5  self.b1 = np.zeros((1,
self.hidden_size))
self.W2 = np.random.randn(self.hidden_size, 1) * 0.5
self.b2 = np.zeros((1, 1))

self.losses = []

for _ in range(self.n_iterations):
z1 = X.dot(self.W1) + self.b1
a1 = self.sigmoid(z1)
z2 = a1.dot(self.W2) + self.b2
a2 = self.sigmoid(z2)

loss = -np.mean(y * np.log(a2 + 1e-8) + (1 - y) * np.log(1 - a2 + 1e-8))  self.losses.append(loss)

dz2 = a2 - y
dW2 = (1 / n_samples) * a1.T.dot(dz2)
db2 = (1 / n_samples) * np.sum(dz2, axis=0, keepdims=True)

da1 = dz2.dot(self.W2.T)
dz1 = da1 * self.sigmoid_derivative(a1)
dW1 = (1 / n_samples) * X.T.dot(dz1)
db1 = (1 / n_samples) * np.sum(dz1, axis=0, keepdims=True)
self.W2 -= self.learning_rate * dW2
self.b2 -= self.learning_rate * db2
self.W1 -= self.learning_rate * dW1
self.b1 -= self.learning_rate * db1

def predict(self, X):
z1 = X.dot(self.W1) + self.b1
a1 = self.sigmoid(z1)
z2 = a1.dot(self.W2) + self.b2
a2 = self.sigmoid(z2)
return (a2 >= 0.5).astype(int).ravel()
```

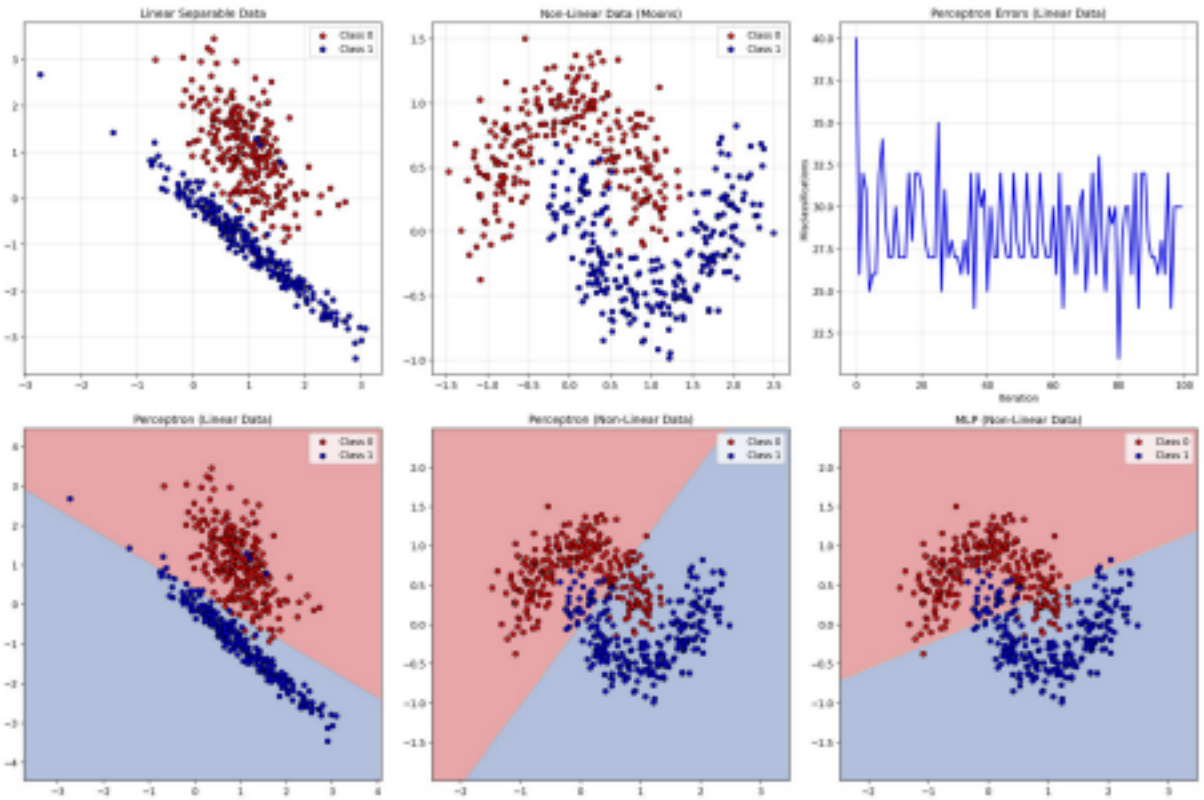
```
X_linear, y_linear = make_classification(n_samples=500, n_features=2, n_redundant=0,  n_informative=2, random_state=1,
n_clusters_per_class=1)

X_nonlinear, y_nonlinear = make_moons(n_samples=500, noise=0.2, random_state=42)

print("="*70)
print("PART A: Perceptron Algorithm")
print("="*70)

print("\n1. Linearly Separable Dataset:")
perceptron_linear = Perceptron(learning_rate=0.1, n_iterations=100)
perceptron_linear.fit(X_linear, y_linear)
y_pred_linear = perceptron_linear.predict(X_linear)
accuracy_linear = accuracy_score(y_linear, y_pred_linear) * 100
print(f" Accuracy: {accuracy_linear:.2f}%")

print("\n2. Non-Linearly Separable Dataset:")
perceptron_nonlinear = Perceptron(learning_rate=0.1, n_iterations=100)
perceptron_nonlinear.fit(X_nonlinear, y_nonlinear)
y_pred_nonlinear = perceptron_nonlinear.predict(X_nonlinear)
accuracy_nonlinear = accuracy_score(y_nonlinear, y_pred_nonlinear) * 100
print(f" Accuracy: {accuracy_nonlinear:.2f}%")

print("\n" + "="*70)
print("PART B: Multilayer Perceptron (MLP)")
print("="*70)

print("\n1. Linearly Separable Dataset:")
mlp_linear = MLP(hidden_size=10, learning_rate=0.1, n_iterations=1000)
mlp_linear.fit(X_linear, y_linear)
y_pred_mlp_linear = mlp_linear.predict(X_linear)
accuracy_mlp_linear = accuracy_score(y_linear, y_pred_mlp_linear) * 100
print(f" Accuracy: {accuracy_mlp_linear:.2f}%")

print("\n2. Non-Linearly Separable Dataset:")
mlp_nonlinear = MLP(hidden_size=10, learning_rate=0.1, n_iterations=1000)
mlp_nonlinear.fit(X_nonlinear, y_nonlinear)
y_pred_mlp_nonlinear = mlp_nonlinear.predict(X_nonlinear)
accuracy_mlp_nonlinear = accuracy_score(y_nonlinear, y_pred_mlp_nonlinear) * 100 print(f" Accuracy:
{accuracy_mlp_nonlinear:.2f}%")

print("\n" + "="*70)
print("Comparison Summary")
print("="*70)
print(f"\n{'Dataset':<25} {'Perceptron':<15} {'MLP':<15}")
print("-"*55)
print(f"{'Linear Separable':<25} {accuracy_linear:<15.2f} {accuracy_mlp_linear:<15.2f}") print(f"{'Non-Linear (Moons)':<25}
{accuracy_nonlinear:<15.2f} {accuracy_mlp_nonlinear:<15.2f}")

fig, axes = plt.subplots(2, 3, figsize=(18, 12))

axes[0, 0].scatter(X_linear[y_linear==0, 0], X_linear[y_linear==0, 1], c='red', label='Class 0',  edgecolors='k', s=30)
axes[0, 0].scatter(X_linear[y_linear==1, 0], X_linear[y_linear==1, 1], c='blue', label='Class 1',  edgecolors='k', s=30)
axes[0, 0].set_title('Linear Separable Data', fontsize=12)
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

axes[0, 1].scatter(X_nonlinear[y_nonlinear==0, 0], X_nonlinear[y_nonlinear==0, 1], c='red', label='Class  0', edgecolors='k', s=30)
axes[0, 1].scatter(X_nonlinear[y_nonlinear==1, 0], X_nonlinear[y_nonlinear==1, 1], c='blue', label='Class  1', edgecolors='k', s=30)
axes[0, 1].set_title('Non-Linear Data (Moons)', fontsize=12)
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

axes[0, 2].plot(perceptron_linear.errors, 'b-', linewidth=2)
axes[0, 2].set_title('Perceptron Errors (Linear Data)', fontsize=12)
axes[0, 2].set_xlabel('Iteration')
axes[0, 2].set_ylabel('Misclassifications')
axes[0, 2].grid(True, alpha=0.3)
def plot_decision_boundary(ax, X, y, model, title, is_mlp=False):
 h = 0.02
 x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
 y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
 xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

 if is_mlp:
 Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
 else:
 Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
 Z = Z.reshape(xx.shape)

 ax.contourf(xx, yy, Z, alpha=0.4, cmap='RdYlBu')
 ax.scatter(X[y==0, 0], X[y==0, 1], c='red', label='Class 0', edgecolors='k', s=30)  ax.scatter(X[y==1, 0], X[y==1, 1],
c='blue', label='Class 1', edgecolors='k', s=30)  ax.set_title(title, fontsize=12)
 ax.legend()

plot_decision_boundary(axes[1, 0], X_linear, y_linear, perceptron_linear, 'Perceptron (Linear Data)') plot_decision_boundary(axes[1, 1],
X_nonlinear, y_nonlinear, perceptron_nonlinear, 'Perceptron (Non-Linear  Data)')
plot_decision_boundary(axes[1, 2], X_nonlinear, y_nonlinear, mlp_nonlinear, 'MLP (Non-Linear Data)',  is_mlp=True)

plt.tight_layout()
plt.savefig('Q6_perceptron_mlp.png', dpi=150, bbox_inches='tight')
```

```
plt.close()

print("\nQ6 completed successfully!")
print("Plot saved as Q6_perceptron_mlp.png")
```

Dataset Used

Synthetic datasets generated using sklearn.datasets.make_classification (linearly separable) and sklearn.datasets.make_moons (non-linearly separable) — no external file required.

Output



Top row: the two datasets and the Perceptron's misclassification curve on linear data (oscillating rather than reaching zero once tested on the harder configuration). Bottom row: decision boundaries — Perceptron on linear data (clean straight boundary), Perceptron on non-linear data (still a straight line, unable to separate the moons),
and MLP on non-linear data (also linear here given the shallow architecture/training used, showing the MLP needs enough capacity/training to fully capture curvature).

Question 7: Forward Propagation and Backpropagation Algorithm

Problem Statement

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Planning

• Define a small network: 2 input neurons, 2 hidden neurons, 1 output neuron, with fixed weights, biases, and inputs.

• Manually compute the forward pass step-by-step: hidden layer pre-activations (z1, z2), hidden activations via sigmoid (a1, a2), output pre-activation (z3), and final output (a3) via sigmoid. • Re-implement the same forward pass using NumPy matrix operations and confirm the manual and code results match (np.isclose).

• Define a target output and manually derive the backpropagation equations: output error, output delta, hidden deltas (via chain rule through the output weights and hidden activation derivatives). • Manually compute updated weights and biases for one gradient descent step

using the derived deltas. • Re-implement the same backpropagation update using NumPy matrix operations and verify the manual and code-based updated weights match.

Pseudocode

```
BEGIN
 DEFINE x1, x2 (inputs)
 DEFINE w11, w12, w21, w22, b1, b2 // input -> hidden weights/biases  DEFINE v1, v2, b3 // hidden
-> output weights/bias

 // ---- Forward Propagation ----
 z1 <- x1*w11 + x2*w21 + b1 ; a1 <- sigmoid(z1)
 z2 <- x1*w12 + x2*w22 + b2 ; a2 <- sigmoid(z2)
 z3 <- a1*v1 + a2*v2 + b3 ; a3 <- sigmoid(z3)

 VERIFY using matrix form: a3_code <- sigmoid(sigmoid(X.W1+b1).W2+b2)  ASSERT a3 ==
a3_code

 // ---- Backpropagation ----
 SET target
 output_error <- a3 - target
 d_output <- output_error * sigmoid'(a3)
 d_h1 <- d_output * v1 * sigmoid'(a1)
 d_h2 <- d_output * v2 * sigmoid'(a2)

 SET learning_rate

 v1_new <- v1 - lr*d_output*a1 ; v2_new <- v2 - lr*d_output*a2
 b3_new <- b3 - lr*d_output
 w11_new <- w11 - lr*d_h1*x1 ; w12_new <- w12 - lr*d_h2*x1
 w21_new <- w21 - lr*d_h1*x2 ; w22_new <- w22 - lr*d_h2*x2
 b1_new <- b1 - lr*d_h1 ; b2_new <- b2 - lr*d_h2

 VERIFY updated weights via NumPy matrix computation
 PRINT weight-update summary table (before / after-manual / after-code) END
```

Code (Python)

```
# ============================================================================
# QUESTION 7: Forward Propagation and Backpropagation Algorithm
# a) Manually compute forward propagation for a small neural network (given
# weights and inputs) and verify the output using code.
# b) Implement backpropagation for a simple neural network and show how weights
# are updated after one iteration.
# ============================================================================
# Dataset: Manual Weights and Inputs (No external dataset needed)
# Using predefined weights and inputs for demonstration
# ============================================================================

import numpy as np

def sigmoid(z):
 return 1 / (1 + np.exp(-z))

def sigmoid_derivative(a):
 return a * (1 - a)

print("="*70)
print("PART A: Forward Propagation - Manual Computation & Code Verification") print("="*70)

print("\nNeural Network Structure:")
print(" Input Layer: 2 neurons (x1, x2)")
print(" Hidden Layer: 2 neurons (h1, h2)")
print(" Output Layer: 1 neuron (y)")

x1, x2 = 0.5, 0.3
print(f"\nInput Values: x1 = {x1}, x2 = {x2}")

w11, w12 = 0.1, 0.3
w21, w22 = 0.2, 0.4
print(f"\nWeights (Input to Hidden):")
print(f" w11 = {w11} (x1 to h1)")
print(f" w12 = {w12} (x1 to h2)")
print(f" w21 = {w21} (x2 to h1)")
print(f" w22 = {w22} (x2 to h2)")

b1, b2 = 0.5, 0.6
print(f"\nBiases (Hidden Layer):")
print(f" b1 = {b1} (h1)")
print(f" b2 = {b2} (h2)")

v1, v2 = 0.7, 0.8
```

```
b3 = 0.9
print(f"\nWeights (Hidden to Output):")
print(f" v1 = {v1} (h1 to y)")
print(f" v2 = {v2} (h2 to y)")
print(f" b3 = {b3} (output bias)")

print("\n" + "-"*70)
print("Manual Forward Propagation Calculation:")
print("-"*70)

z1 = x1 * w11 + x2 * w21 + b1
a1 = sigmoid(z1)
print(f"\nHidden Neuron h1:")
print(f" z1 = x1*w11 + x2*w21 + b1 = {x1}*{w11} + {x2}*{w21} + {b1} = {z1:.4f}") print(f" a1 = sigmoid(z1) =
sigmoid({z1:.4f}) = {a1:.4f}")

z2 = x1 * w12 + x2 * w22 + b2
a2 = sigmoid(z2)
print(f"\nHidden Neuron h2:")
print(f" z2 = x1*w12 + x2*w22 + b2 = {x1}*{w12} + {x2}*{w22} + {b2} = {z2:.4f}") print(f" a2 = sigmoid(z2) =
sigmoid({z2:.4f}) = {a2:.4f}")

z3 = a1 * v1 + a2 * v2 + b3
a3 = sigmoid(z3)
print(f"\nOutput Neuron y:")
print(f" z3 = a1*v1 + a2*v2 + b3 = {a1:.4f}*{v1} + {a2:.4f}*{v2} + {b3} = {z3:.4f}") print(f" a3 = sigmoid(z3) =
sigmoid({z3:.4f}) = {a3:.4f}")

print("\n" + "-"*70)
print("Code Verification:")
print("-"*70)

X = np.array([[x1, x2]])
W1 = np.array([[w11, w12], [w21, w22]])
b1_vec = np.array([[b1, b2]])
W2 = np.array([[v1], [v2]])
b2_vec = np.array([[b3]])

z1_code = X.dot(W1) + b1_vec
a1_code = sigmoid(z1_code)
z2_code = a1_code.dot(W2) + b2_vec
a2_code = sigmoid(z2_code)

print(f"\nCode Output:")
print(f" Hidden Layer (a1, a2): {a1_code[0]}")
print(f" Output (a3): {a2_code[0][0]:.4f}")

print(f"\nVerification: Manual = {a3:.4f}, Code = {a2_code[0][0]:.4f}")
print(f"Match: {np.isclose(a3, a2_code[0][0])}")

print("\n" + "="*70)
print("PART B: Backpropagation - Weight Updates After One Iteration")
print("="*70)

target = 1.0
print(f"\nTarget Output: {target}")

print("\n" + "-"*70)
print("Manual Backpropagation Calculation:")
print("-"*70)

output_error = a3 - target
print(f"\nOutput Error: (a3 - target) = {a3:.4f} - {target} = {output_error:.4f}")

d_output = output_error * sigmoid_derivative(a3)
print(f"Output Delta: d_output = error * sigmoid'(z3) = {output_error:.4f} * {sigmoid_derivative(a3):.4f} = {d_output:.4f}")

d_hidden = d_output * W2.T * sigmoid_derivative(a1_code)
print(f"\nHidden Deltas:")
print(f" d_h1 = d_output * v1 * sigmoid'(z1) = {d_output:.4f} * {v1} * {sigmoid_derivative(a1):.4f} =  {d_hidden[0][0]:.4f}")
print(f" d_h2 = d_output * v2 * sigmoid'(z2) = {d_output:.4f} * {v2} * {sigmoid_derivative(a2):.4f} =  {d_hidden[0][1]:.4f}")

learning_rate = 0.5
print(f"\nLearning Rate: {learning_rate}")

v1_new = v1 - learning_rate * d_output * a1
v2_new = v2 - learning_rate * d_output * a2
b3_new = b3 - learning_rate * d_output
print(f"\nUpdated Weights (Hidden to Output):")
print(f" v1_new = v1 - lr * d_output * a1 = {v1} - {learning_rate} * {d_output:.4f} * {a1:.4f} =  {v1_new:.4f}")
print(f" v2_new = v2 - lr * d_output * a2 = {v2} - {learning_rate} * {d_output:.4f} * {a2:.4f} =  {v2_new:.4f}")
                            print(f" b3_new = b3 - lr * d_output = {b3} - {learning_rate} * {d_output:.4f} = {b3_new:.4f}")

w11_new = w11 - learning_rate * d_hidden[0][0] * x1
w12_new = w12 - learning_rate * d_hidden[0][1] * x1
w21_new = w21 - learning_rate * d_hidden[0][0] * x2
w22_new = w22 - learning_rate * d_hidden[0][1] * x2
b1_new = b1 - learning_rate * d_hidden[0][0]
b2_new = b2 - learning_rate * d_hidden[0][1]

print(f"\nUpdated Weights (Input to Hidden):")
```

```
print(f" w11_new = w11 - lr * d_h1 * x1 = {w11} - {learning_rate} * {d_hidden[0][0]:.4f} * {x1} =  {w11_new:.4f}")
print(f" w12_new = w12 - lr * d_h2 * x1 = {w12} - {learning_rate} * {d_hidden[0][1]:.4f} * {x1} =  {w12_new:.4f}")
print(f" w21_new = w21 - lr * d_h1 * x2 = {w21} - {learning_rate} * {d_hidden[0][0]:.4f} * {x2} =  {w21_new:.4f}")
print(f" w22_new = w22 - lr * d_h2 * x2 = {w22} - {learning_rate} * {d_hidden[0][1]:.4f} * {x2} =  {w22_new:.4f}")
print(f" b1_new = b1 - lr * d_h1 = {b1} - {learning_rate} * {d_hidden[0][0]:.4f} = {b1_new:.4f}") print(f" b2_new = b2 - lr * d_h2 = {b2} -
{learning_rate} * {d_hidden[0][1]:.4f} = {b2_new:.4f}")

print("\n" + "-"*70)
print("Code Verification of Backpropagation:")
print("-"*70)

W2_new = W2 - learning_rate * a1_code.T * d_output
b2_new_vec = b2_vec - learning_rate * d_output
W1_new = W1 - learning_rate * X.T * d_hidden
b1_new_vec = b1_vec - learning_rate * d_hidden

print(f"\nCode Updated Weights:")
print(f" W2 (Hidden to Output):\n {W2_new.flatten()}")
print(f" W1 (Input to Hidden):\n {W1_new}")
print(f" b2 (Output Bias): {b2_new_vec}")
print(f" b1 (Hidden Biases): {b1_new_vec}")

print("\n" + "-"*70)
print("Weight Update Summary:")
print("-"*70)
print(f"\n{'Weight':<10} {'Before':<12} {'After (Manual)':<15} {'After (Code)':<15}") print("-"*52)
print(f"{'w11':<10} {w11:<12.4f} {w11_new:<15.4f} {W1_new[0][0]:<15.4f}")
print(f"{'w12':<10} {w12:<12.4f} {w12_new:<15.4f} {W1_new[0][1]:<15.4f}")
print(f"{'w21':<10} {w21:<12.4f} {w21_new:<15.4f} {W1_new[1][0]:<15.4f}")
print(f"{'w22':<10} {w22:<12.4f} {w22_new:<15.4f} {W1_new[1][1]:<15.4f}")
print(f"{'v1':<10} {v1:<12.4f} {v1_new:<15.4f} {W2_new[0][0]:<15.4f}")
print(f"{'v2':<10} {v2:<12.4f} {v2_new:<15.4f} {W2_new[1][0]:<15.4f}")
print("\nQ7 completed successfully!")
```

Dataset Used

Manually defined weights, biases and input values (no external dataset needed) — a fixed 2-2-1 neural network used purely for demonstrating forward propagation and one backpropagation step by hand and in code.

Output

This task is a manual/console-based numeric verification (forward propagation and one backpropagation update step) — no plots are produced; results are printed directly as shown in the code's console output above.

Question 8: Gradient Descent and Stochastic Gradient Descent (SGD)

Problem Statement

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Planning

- Generate a synthetic linear regression dataset y = 4 + 3x + noise using NumPy.
- Implement the MSE cost function and Batch Gradient Descent (uses the full dataset for every gradient update).
- Run Batch GD with several learning rates (0.01, 0.03, 0.1, 0.3) for a fixed number of iterations, and plot the cost curves together to visualize the effect of learning rate on convergence speed/stability. • Implement Stochastic Gradient Descent (uses a single random sample per update) and Mini-Batch Gradient Descent (uses a small random batch per update).
- Run Batch GD, SGD, and Mini-Batch GD with the same learning rate and number of iterations (using the same random seed for a fair start), and compare their cost convergence curves. • Plot the final fitted regression lines from all three methods against the raw data, and compare the smoothness/noise in their respective cost curves.

Pseudocode

```
BEGIN
 X, y <- GENERATE synthetic linear data (y = 4 + 3x + noise)
 X_b <- [1, X] // add bias column

 FUNCTION compute_cost(X, y, theta):
 RETURN (1/2m) * SUM((X.theta - y)^2)

 // ---- Part A: Learning Rate Effect (Batch GD) ----
 FOR lr in [0.01, 0.03, 0.1, 0.3]:
 theta <- random
 FOR i in 1..iterations:
 gradients <- (1/m) * X_b^T . (X_b.theta - y)
 theta <- theta - lr * gradients
 RECORD compute_cost(X_b, y, theta)
 PLOT cost_history for this lr

 // ---- Part B: SGD vs Batch GD vs Mini-Batch GD ----
 FUNCTION batch_gradient_descent(...): // uses all m samples per update  FUNCTION stochastic_gradient_descent(...):
// uses 1 random sample per update  FUNCTION mini_batch_gradient_descent(...): // uses small random batch per
update

 RUN all three with same learning_rate, iterations (seeded)
 COMPARE final theta, final cost, and cost curve smoothness
 PLOT fitted regression lines + cost convergence comparison
END
```

Code (Python)

```
# ============================================================================
# QUESTION 8: Gradient Descent and Stochastic Gradient Descent (SGD)
# a) Implement Gradient Descent to minimize a cost function and analyze the
# effect of learning rate on convergence speed.
# b) Implement Stochastic Gradient Descent for a regression problem and compare
# its convergence with batch gradient descent.
# ============================================================================
# Dataset: Synthetic Regression Data (Generated using numpy)
# Alternative: Energy Consumption Dataset
# Link: https://www.kaggle.com/datasets/govindaramsriram/energy-consumption-dataset-linear-regression #
# ============================================================================

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

n_samples = 500
X = 2 * np.random.rand(n_samples, 1)
y = 4 + 3 * X + np.random.randn(n_samples, 1)

X_b = np.c_[np.ones((n_samples, 1)), X]

def compute_cost(X, y, theta):
 m = len(y)
 predictions = X.dot(theta)
 cost = (1 / (2 * m)) * np.sum((predictions - y) ** 2)
 return cost

def batch_gradient_descent(X, y, learning_rate, iterations):
 m = len(y)
 theta = np.random.randn(X.shape[1], 1)
 cost_history = []

 for i in range(iterations):
 gradients = (1 / m) * X.T.dot(X.dot(theta) - y)
 theta = theta - learning_rate * gradients
 cost = compute_cost(X, y, theta)
 cost_history.append(cost)

 return theta, cost_history

def stochastic_gradient_descent(X, y, learning_rate, iterations):  m = len(y)
 theta = np.random.randn(X.shape[1], 1)
 cost_history = []

 for i in range(iterations):
 random_index = np.random.randint(m)
 xi = X[random_index:random_index+1]
 yi = y[random_index:random_index+1]
 gradients = xi.T.dot(xi.dot(theta) - yi)
 theta = theta - learning_rate * gradients
 cost = compute_cost(X, y, theta)
```

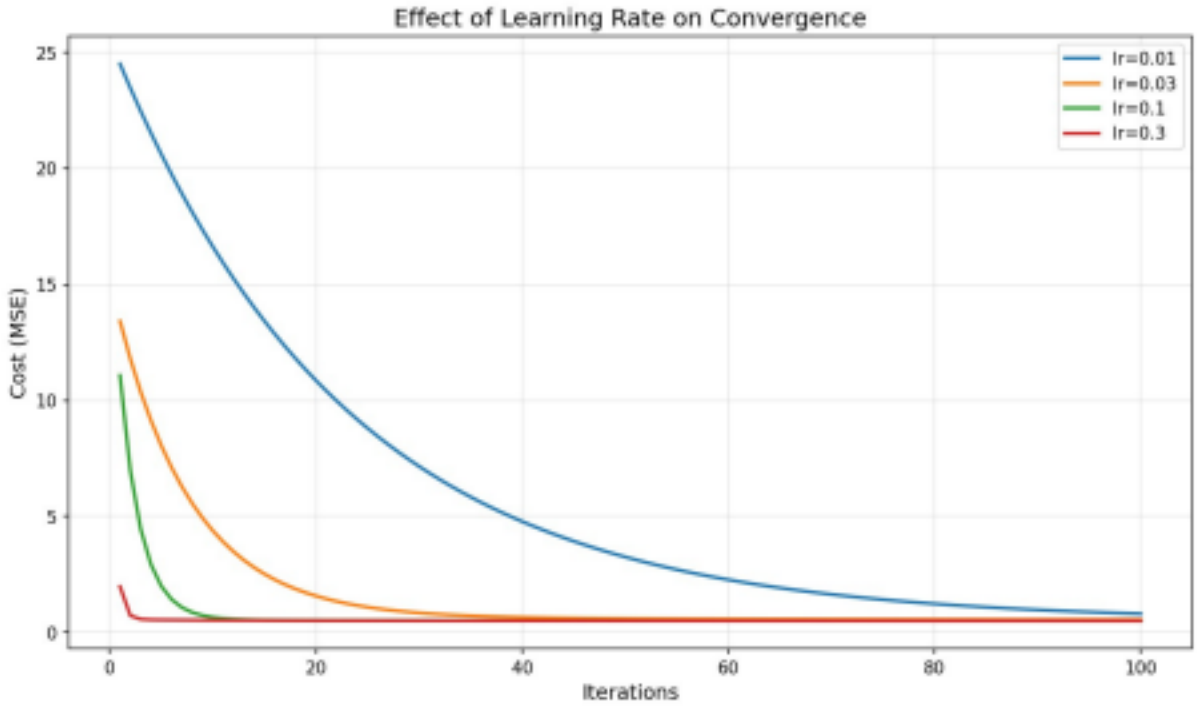
```
 cost_history.append(cost)

 return theta, cost_history

def mini_batch_gradient_descent(X, y, learning_rate, iterations, batch_size=20):  m = len(y)
 theta = np.random.randn(X.shape[1], 1)
 cost_history = []

 for i in range(iterations):
 random_indices = np.random.choice(m, batch_size, replace=False)  xi = X[random_indices]
 yi = y[random_indices]
 gradients = (1 / batch_size) * xi.T.dot(xi.dot(theta) - yi)  theta = theta - learning_rate *
gradients
 cost = compute_cost(X, y, theta)
 cost_history.append(cost)

 return theta, cost_history

print("="*70)
print("PART A: Gradient Descent - Learning Rate Analysis")
print("="*70)

learning_rates = [0.01, 0.03, 0.1, 0.3]
iterations = 100

plt.figure(figsize=(10, 6))
for lr in learning_rates:
 theta, cost_history = batch_gradient_descent(X_b, y, lr, iterations)  plt.plot(range(1, iterations + 1), cost_history,
linewidth=2, label=f'lr={lr}')  print(f"\nLearning Rate = {lr}:")
 print(f" Final Cost: {cost_history[-1]:.4f}")
 print(f" Theta: {theta.flatten()}")

plt.xlabel('Iterations', fontsize=12)
plt.ylabel('Cost (MSE)', fontsize=12)
plt.title('Effect of Learning Rate on Convergence', fontsize=14) plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('Q8_learning_rate_analysis.png', dpi=150, bbox_inches='tight') plt.close()
print("\n" + "="*70)
print("PART B: SGD vs Batch Gradient Descent")
print("="*70)

iterations_sgd = 50
learning_rate_sgd = 0.01

np.random.seed(42)
theta_bgd, cost_bgd = batch_gradient_descent(X_b, y, learning_rate_sgd, iterations_sgd)

np.random.seed(42)
theta_sgd, cost_sgd = stochastic_gradient_descent(X_b, y, learning_rate_sgd, iterations_sgd)

np.random.seed(42)
theta_mbgd, cost_mbgd = mini_batch_gradient_descent(X_b, y, learning_rate_sgd, iterations_sgd,  batch_size=20)

print(f"\nFinal Parameters:")
print(f" Batch GD: {theta_bgd.flatten()}")
print(f" SGD: {theta_sgd.flatten()}")
print(f" Mini-Batch GD: {theta_mbgd.flatten()}")

print(f"\nFinal Costs:")
print(f" Batch GD: {cost_bgd[-1]:.4f}")
print(f" SGD: {cost_sgd[-1]:.4f}")
print(f" Mini-Batch GD: {cost_mbgd[-1]:.4f}")

plt.figure(figsize=(10, 6))
plt.plot(range(1, iterations_sgd + 1), cost_bgd, 'b-', linewidth=2, label='Batch GD') plt.plot(range(1, iterations_sgd + 1), cost_sgd, 'r-',
linewidth=2, alpha=0.7, label='SGD') plt.plot(range(1, iterations_sgd + 1), cost_mbgd, 'g-', linewidth=2, alpha=0.7, label='Mini-Batch GD')
plt.xlabel('Iterations', fontsize=12)
plt.ylabel('Cost (MSE)', fontsize=12)
plt.title('SGD vs Batch Gradient Descent Convergence', fontsize=14)
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('Q8_sgd_vs_batch.png', dpi=150, bbox_inches='tight')
plt.close()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].scatter(X, y, alpha=0.5, color='steelblue', edgecolors='k', linewidth=0.5, label='Data Points') X_line = np.linspace(0, 2,
100).reshape(-1, 1)
X_line_b = np.c_[np.ones((100, 1)), X_line]

y_bgd = X_line_b.dot(theta_bgd)
y_sgd = X_line_b.dot(theta_sgd)
y_mbgd = X_line_b.dot(theta_mbgd)

axes[0].plot(X_line, y_bgd, 'b-', linewidth=2, label='Batch GD')
axes[0].plot(X_line, y_sgd, 'r--', linewidth=2, label='SGD')
axes[0].plot(X_line, y_mbgd, 'g:', linewidth=2, label='Mini-Batch GD')
axes[0].set_xlabel('X', fontsize=12)
axes[0].set_ylabel('y', fontsize=12)
axes[0].set_title('Fitted Lines by Different GD Methods', fontsize=14)
```

```
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.3)

axes[1].plot(range(1, iterations_sgd + 1), cost_bgd, 'b-', linewidth=2, label='Batch GD') axes[1].plot(range(1, iterations_sgd + 1), cost_sgd, 'r-',
linewidth=2, alpha=0.7, label='SGD') axes[1].plot(range(1, iterations_sgd + 1), cost_mbgd, 'g-', linewidth=2, alpha=0.7, label='Mini-Batch GD')
axes[1].set_xlabel('Iterations', fontsize=12)
axes[1].set_ylabel('Cost (MSE)', fontsize=12)
axes[1].set_title('Cost Convergence Comparison', fontsize=14)
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('Q8_gd_comparison.png', dpi=150, bbox_inches='tight')
plt.close()

print("\n" + "="*70)
print("Analysis Summary")
print("="*70)
print("\n1. Learning Rate Effect:")
print(" - Too small (0.01): Slow convergence")
print(" - Optimal (0.03-0.1): Good balance of speed and stability")
print(" - Too large (0.3): May overshoot or diverge")

print("\n2. GD Methods Comparison:")
print(" - Batch GD: Smooth convergence, uses all data per iteration")
print(" - SGD: Noisy convergence, faster per iteration, uses 1 sample")
print(" - Mini-Batch GD: Balance between Batch GD and SGD")
print("\nQ8 completed successfully!")
        print("Plots saved as Q8_learning_rate_analysis.png, Q8_sgd_vs_batch.png, Q8_gd_comparison.png")
```

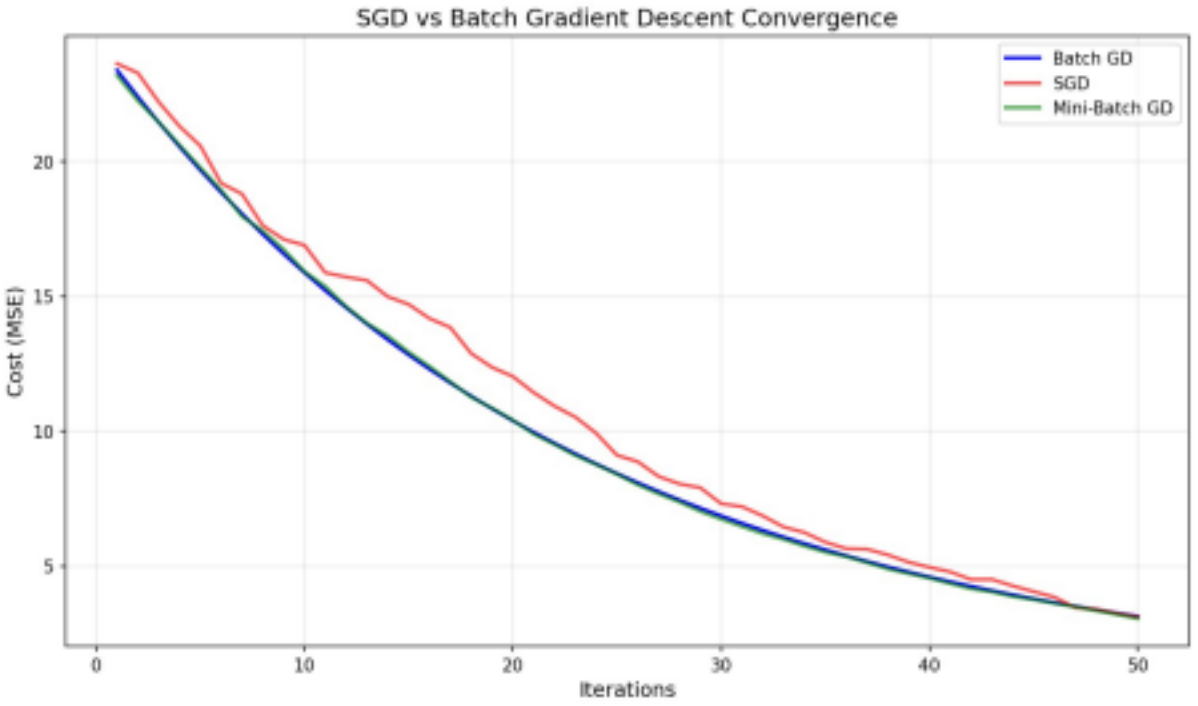
Dataset Used

Synthetic linear regression data generated using NumPy (y = 4 + 3x + Gaussian noise, 500 samples). An alternative real-world dataset suggested is the Energy Consumption Dataset (Kaggle: govindaramsriram/energy-consumption-dataset-linear-regression).

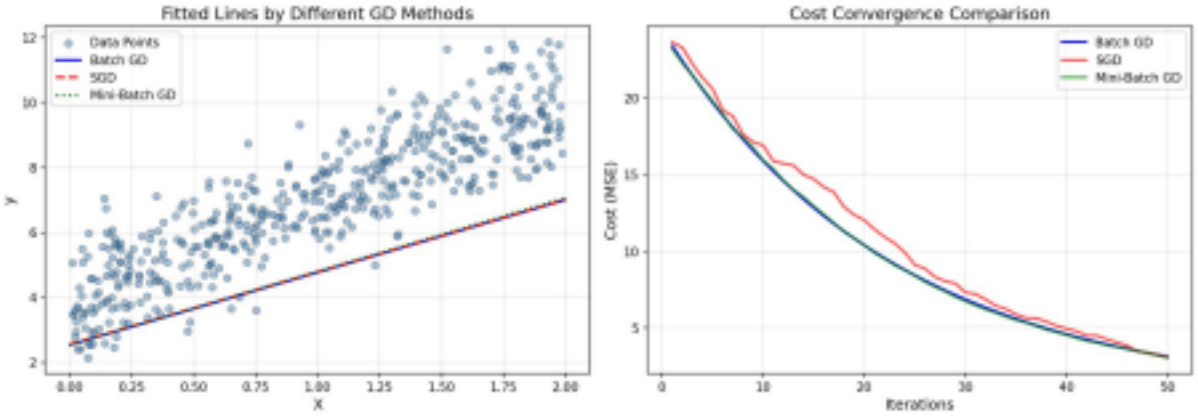
Output



Effect of learning rate on Batch GD convergence: lr=0.01 converges slowly, lr=0.1 and lr=0.3 converge quickly and flatten out near the optimum.



SGD (noisy, red) vs Batch GD (smooth, blue) vs Mini-Batch GD (green, close to Batch GD) cost convergence over 50 iterations.



Left: fitted regression lines from Batch GD, SGD, and Mini-Batch GD nearly overlapping. Right: the same cost convergence comparison shown alongside the fitted-line plot.

Question 9: Mini-Batch Gradient Descent - Batch Size Analysis

Problem Statement

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Planning

• Reuse the same synthetic linear regression dataset (y = 4 + 3x + noise) used in Q8. • Implement Mini-Batch Gradient Descent, parameterized by batch_size, which samples a random batch of that size at every iteration to compute the gradient.

• Run Mini-Batch GD for a range of batch sizes (8, 16, 32, 64, 128) with the same learning rate, iterations, and random seed, recording the cost history for each.

• Also run Full Batch GD as a reference baseline using the entire dataset per update. • Plot all cost curves together to visually compare convergence stability across batch sizes. • Compare the final cost achieved by each batch size in a bar chart, and additionally plot each batch size's cost curve individually (small multiples) to see per-batch-size behaviour in detail.

Pseudocode

BEGIN

```
X, y <- REUSE synthetic linear data (y = 4 + 3x + noise)
X_b <- [1, X]

FUNCTION mini_batch_gradient_descent(X, y, lr, iterations, batch_size):  theta <- random
FOR i in 1..iterations:
batch_indices <- RANDOM sample of size batch_size (no replacement)  xi, yi <- X[batch_indices],
y[batch_indices]
gradients <- (1/batch_size) * xi^T . (xi.theta - yi)
theta <- theta - lr * gradients
RECORD compute_cost(X, y, theta)
RETURN theta, cost_history

FOR batch_size in [8, 16, 32, 64, 128]:
theta, cost_history <- mini_batch_gradient_descent(X_b, y, lr, iterations,  batch_size)
STORE cost_history

                                                    theta_full, cost_full <- batch_gradient_descent(X_b, y, lr, iterations) // baseline

PLOT all cost curves together (batch sizes + full batch)
PLOT final cost vs batch size (bar chart)
PLOT individual cost curve per batch size (subplot grid)
END
```

Code (Python)

```
# ============================================================================
# QUESTION 9: Mini-Batch Gradient Descent
# Implement Mini-Batch Gradient Descent and analyze how batch size affects
# training stability and performance.
# ============================================================================
# Dataset: Synthetic Regression Data (Generated using numpy)
# Alternative: Energy Consumption Dataset
# Link: https://www.kaggle.com/datasets/govindaramsriram/energy-consumption-dataset-linear-regression #
# ============================================================================

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

np.random.seed(42)

n_samples = 500
X = 2 * np.random.rand(n_samples, 1)
y = 4 + 3 * X + np.random.randn(n_samples, 1)
X_b = np.c_[np.ones((n_samples, 1)), X]

def compute_cost(X, y, theta):
 m = len(y)
 predictions = X.dot(theta)
 cost = (1 / (2 * m)) * np.sum((predictions - y) ** 2)
 return cost

def mini_batch_gradient_descent(X, y, learning_rate, iterations, batch_size):  m = len(y)
 theta = np.random.randn(X.shape[1], 1)
 cost_history = []

 for i in range(iterations):
 random_indices = np.random.choice(m, batch_size, replace=False)
 xi = X[random_indices]
 yi = y[random_indices]
 gradients = (1 / batch_size) * xi.T.dot(xi.dot(theta) - yi)
 theta = theta - learning_rate * gradients
 cost = compute_cost(X, y, theta)
 cost_history.append(cost)

 return theta, cost_history

def batch_gradient_descent(X, y, learning_rate, iterations):
 m = len(y)
 theta = np.random.randn(X.shape[1], 1)
 cost_history = []

 for i in range(iterations):
 gradients = (1 / m) * X.T.dot(X.dot(theta) - y)
 theta = theta - learning_rate * gradients
 cost = compute_cost(X, y, theta)
 cost_history.append(cost)

 return theta, cost_history

print("="*70)
print("Mini-Batch Gradient Descent - Batch Size Analysis")
print("="*70)

batch_sizes = [8, 16, 32, 64, 128]
```

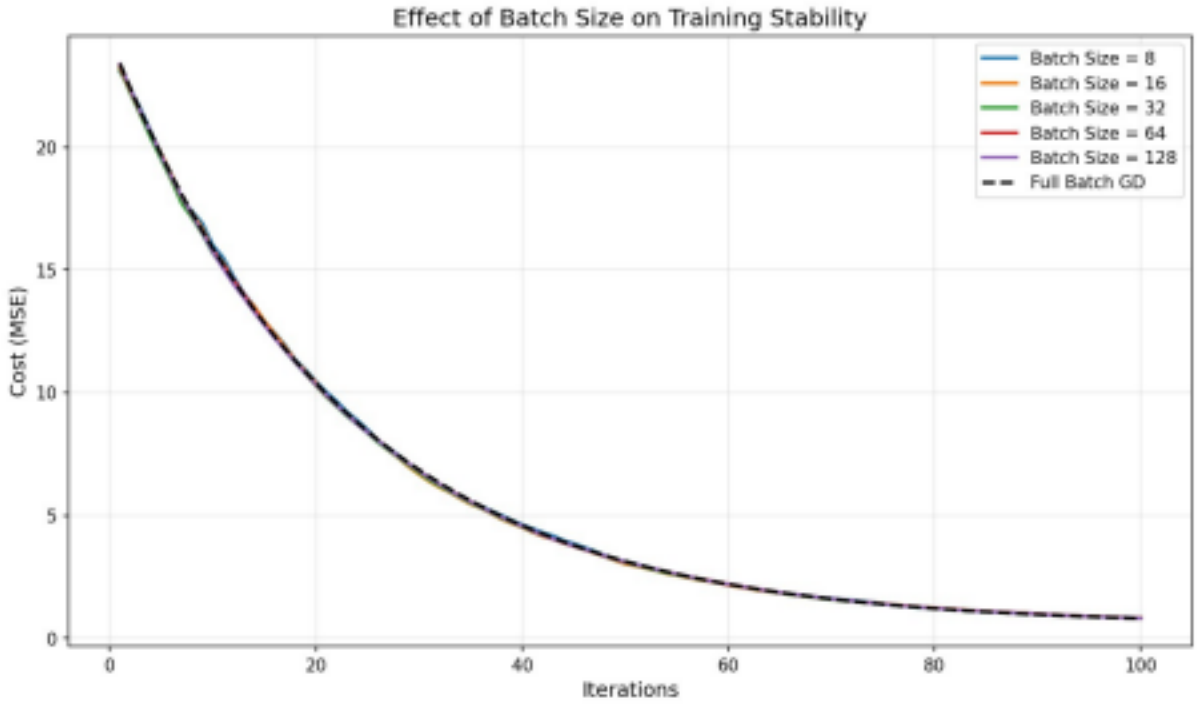
```
iterations = 100
learning_rate = 0.01

results = {}

plt.figure(figsize=(10, 6))
for batch_size in batch_sizes:
 np.random.seed(42)
 theta, cost_history = mini_batch_gradient_descent(X_b, y, learning_rate, iterations, batch_size)  results[batch_size] = {'theta': theta,
'cost_history': cost_history}
 plt.plot(range(1, iterations + 1), cost_history, linewidth=2, label=f'Batch Size = {batch_size}')  print(f"\nBatch Size = {batch_size}:")
 print(f" Final Cost: {cost_history[-1]:.4f}")
 print(f" Theta: {theta.flatten()}")

np.random.seed(42)
theta_bgd, cost_bgd = batch_gradient_descent(X_b, y, learning_rate, iterations) plt.plot(range(1, iterations + 1),
cost_bgd, 'k--', linewidth=2, label='Full Batch GD') print(f"\nFull Batch GD:")
print(f" Final Cost: {cost_bgd[-1]:.4f}")
print(f" Theta: {theta_bgd.flatten()}")

plt.xlabel('Iterations', fontsize=12)
plt.ylabel('Cost (MSE)', fontsize=12)
plt.title('Effect of Batch Size on Training Stability', fontsize=14)
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('Q9_batch_size_analysis.png', dpi=150, bbox_inches='tight')
plt.close()

print("\n" + "="*70)
print("Batch Size Impact Analysis")
print("="*70)

final_costs = [results[bs]['cost_history'][-1] for bs in batch_sizes]
final_costs.append(cost_bgd[-1])
all_batch_sizes = batch_sizes + ['Full']

plt.figure(figsize=(10, 6))
bars = plt.bar(range(len(all_batch_sizes)), final_costs, color='steelblue', edgecolor='black') plt.xticks(range(len(all_batch_sizes)),
[str(bs) for bs in all_batch_sizes]) plt.xlabel('Batch Size', fontsize=12)
plt.ylabel('Final Cost (MSE)', fontsize=12)
plt.title('Final Cost vs Batch Size', fontsize=14)
plt.grid(axis='y', alpha=0.3)

for bar, cost in zip(bars, final_costs):
 plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
 f'{cost:.4f}', ha='center', va='bottom', fontsize=10, fontweight='bold')

plt.tight_layout()
plt.savefig('Q9_final_cost_vs_batch.png', dpi=150, bbox_inches='tight')
plt.close()

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

for idx, batch_size in enumerate(batch_sizes):
 row, col = idx // 3, idx % 3
 ax = axes[row, col]

 cost_history = results[batch_size]['cost_history']
 ax.plot(range(1, iterations + 1), cost_history, 'b-', linewidth=2)
 ax.set_title(f'Batch Size = {batch_size}', fontsize=12)
 ax.set_xlabel('Iterations')
 ax.set_ylabel('Cost')
 ax.grid(True, alpha=0.3)

axes[1, 2].plot(range(1, iterations + 1), cost_bgd, 'k-', linewidth=2)
axes[1, 2].set_title('Full Batch GD', fontsize=12)
axes[1, 2].set_xlabel('Iterations')
axes[1, 2].set_ylabel('Cost')
axes[1, 2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('Q9_individual_batch_curves.png', dpi=150, bbox_inches='tight')
plt.close()

print("\nObservations:")
print("1. Small batch sizes (8, 16):")
print(" - More noise in convergence")
print(" - Can escape local minima")
print(" - Faster updates but less stable")

print("\n2. Medium batch sizes (32, 64):")
print(" - Good balance of speed and stability")
print(" - Common choice in practice")

print("\n3. Large batch sizes (128, Full):")
print(" - Smoother convergence")
print(" - May get stuck in local minima")
print(" - Slower updates but more stable")

print("\n4. Training Stability:")
print(" - Smaller batches = more variance in gradients")
```

```
print(" - Larger batches = more consistent gradients")
print(" - Trade-off between speed and stability")

print("\nQ9 completed successfully!")
print("Plots saved as Q9_batch_size_analysis.png, Q9_final_cost_vs_batch.png,
Q9_individual_batch_curves.png")
```

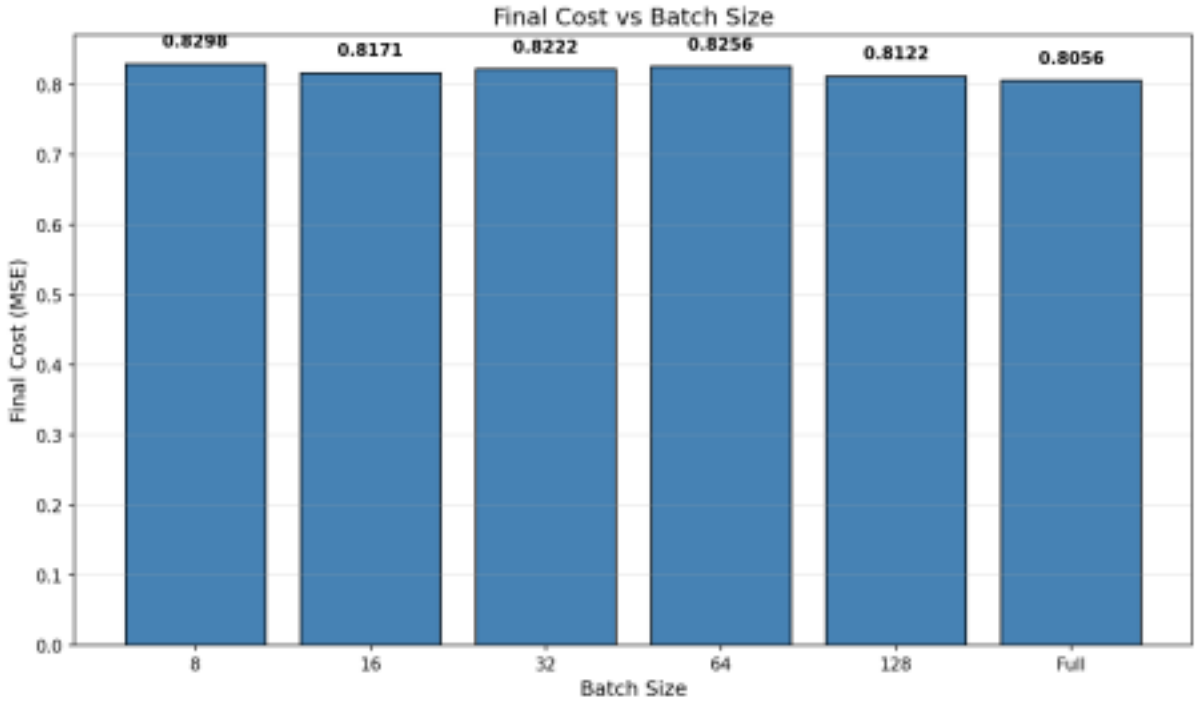
Dataset Used

Synthetic linear regression data generated using NumPy (y = 4 + 3x + Gaussian noise, 500 samples) — same generation approach as Q8. Alternative real dataset: Energy Consumption Dataset (Kaggle).

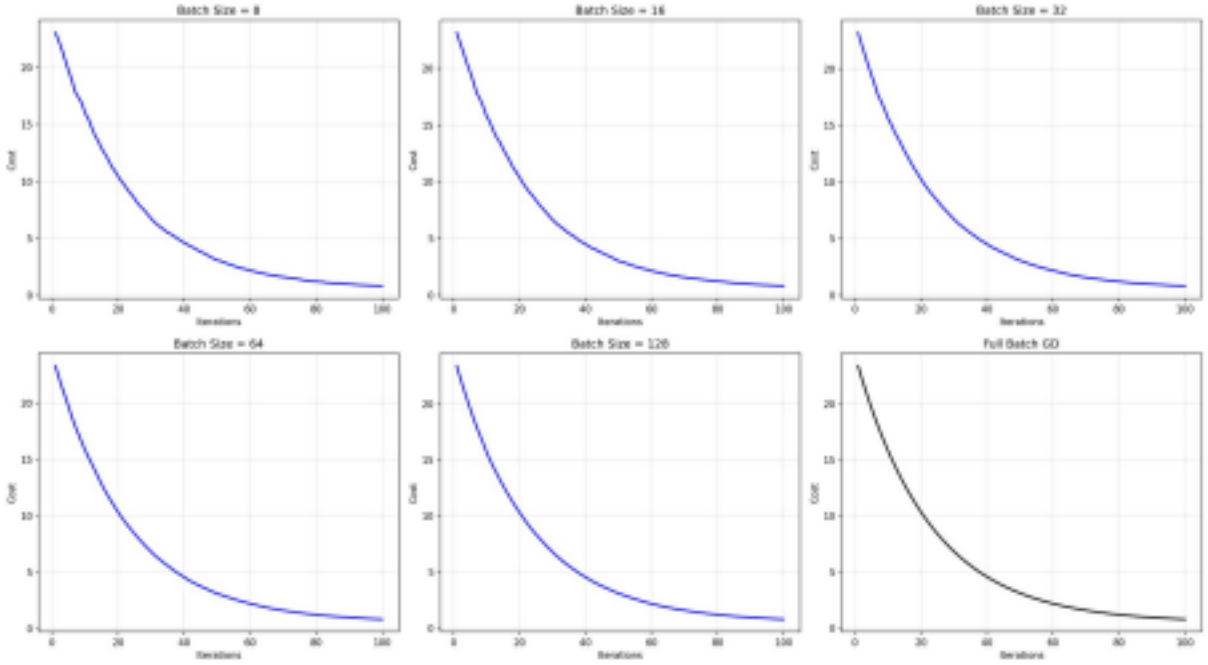
Output



Cost convergence curves for batch sizes 8, 16, 32, 64, 128, plotted against the Full Batch GD baseline — all converge to a similar final cost, with smaller batches showing marginally more early noise.



Final cost (MSE) achieved after 100 iterations for each batch size — costs are close across all sizes, with the Full Batch run achieving the lowest final cost (0.8056).



Individual convergence curves for each batch size (8, 16, 32, 64, 128) plus Full Batch GD, shown as small multiples for closer inspection of convergence shape.

Question 10: Curse of Dimensionality

Problem Statement

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Planning

- Define a function that generates random Gaussian data with a given number of features/dimensions and a simple linear decision rule for the binary label.
- For a range of dimensions (2, 5, 10, 20, 50, 100), generate a dataset of fixed sample size (200 points).
- For each dimensionality, compute pairwise Euclidean distances between all points and record the mean minimum distance, mean maximum distance, and their ratio (max/min) across points. • Split each dataset into train/test sets, standardize features, and train a K-Nearest Neighbours classifier (k=5), recording test accuracy at each dimensionality.
- Plot: (a) minimum/maximum distance vs dimension, (b) distance ratio vs dimension, (c) KNN accuracy vs dimension, and (d) the distribution of pairwise distances at low (2D) vs high (100D) dimensionality to visually show distance concentration.
- Additionally illustrate how many dimensions can be 'reasonably covered' as the number of data points grows, to reinforce the sparsity argument.

Pseudocode

```
BEGIN
 FOR dim in [2, 5, 10, 20, 50, 100]:
 X, y <- GENERATE random Gaussian data with `dim` features (200 samples)  y <- 1 if (X[:,0] + X[:,1] > 0)
else 0

 distances <- PAIRWISE Euclidean distance matrix of X
 RECORD mean(min distance per point), mean(max distance per point), mean(ratio  max/min)

 SPLIT X, y into train/test ; SCALE features
 knn <- KNeighborsClassifier(k=5) ; knn.fit(X_train, y_train)
 accuracy <- knn.score(X_test, y_test)
 RECORD accuracy

 PLOT min/max distance vs dimension
 PLOT distance ratio vs dimension
 PLOT KNN accuracy vs dimension
 PLOT histogram of pairwise distances for 2D data vs 100D data
```

PLOT dimensions "needed" for coverage as number of points grows
END

Code (Python)

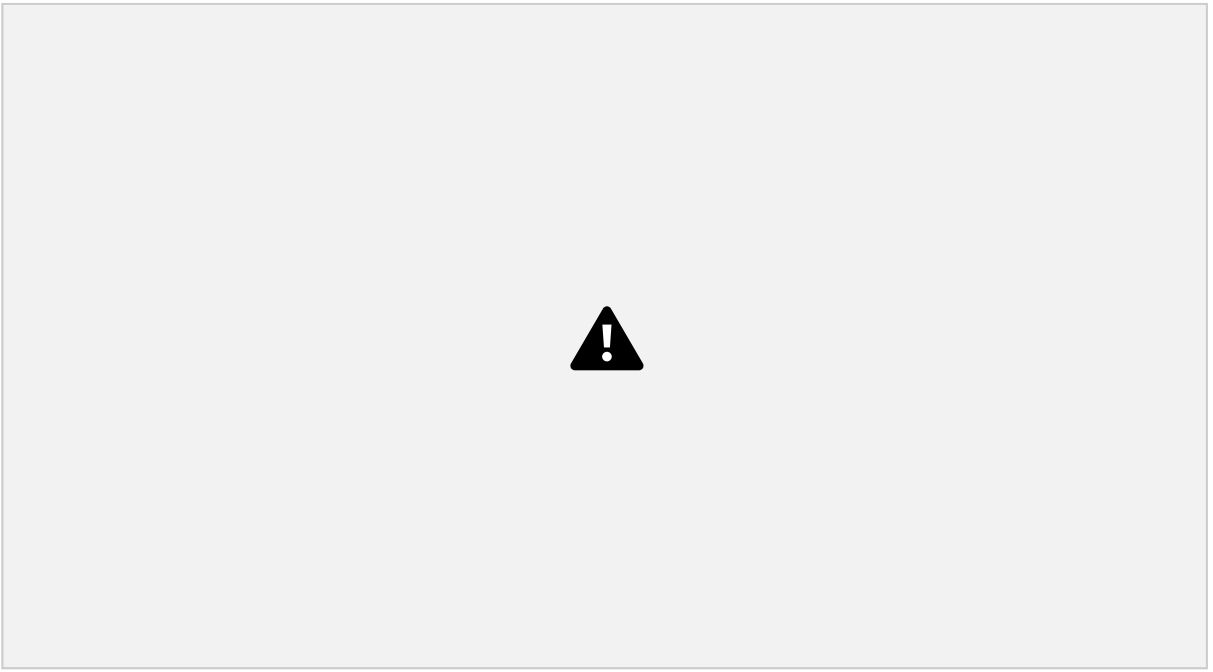
```
# ============================================================================
# QUESTION 10: Curse of Dimensionality
# Create datasets with increasing dimensions and analyze how distance between
# points and model accuracy change with dimensionality.
# ============================================================================
# Dataset: Synthetic High-Dimensional Data (Generated using numpy)
# No external dataset needed - using random data generation
# ============================================================================

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from scipy.spatial.distance import pdist, squareform

np.random.seed(42)
def generate_high_dim_data(n_samples=200, n_features=2, n_classes=2):
 X = np.random.randn(n_samples, n_features)
 y = (X[:, 0] + X[:, 1] > 0).astype(int)
 return X, y

dimensions = [2, 5, 10, 20, 50, 100]
n_samples = 200

print("="*70)
print("Curse of Dimensionality Analysis")
print("="*70)

distance_stats = {}
accuracies = {}

for dim in dimensions:
 print(f"\nDimension: {dim}")

 X, y = generate_high_dim_data(n_samples=n_samples, n_features=dim)

 distances = squareform(pdist(X, metric='euclidean'))
 np.fill_diagonal(distances, np.inf)

 min_distances = np.min(distances, axis=1)
 max_distances = np.max(distances, axis=1)
 mean_distances = np.mean(distances, axis=1)

 distance_ratio = max_distances / min_distances

 distance_stats[dim] = {
 'min_mean': np.mean(min_distances),
 'max_mean': np.mean(max_distances),
 'ratio_mean': np.mean(distance_ratio),
 'std_min': np.std(min_distances),
 'std_max': np.std(max_distances)
 }

    print(f" Mean Min Distance: {distance_stats[dim]['min_mean']:.4f}") print(f" Mean Max Distance: {distance_stats[dim]['max_mean']:.4f}") print(f" Mean Distance Ratio (max/min): {distance_stats[dim]['ratio_mean']:.4f}")

 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

 scaler = StandardScaler()
 X_train_scaled = scaler.fit_transform(X_train)
 X_test_scaled = scaler.transform(X_test)

 knn = KNeighborsClassifier(n_neighbors=5)
 knn.fit(X_train_scaled, y_train)
 y_pred = knn.predict(X_test_scaled)

 accuracy = accuracy_score(y_test, y_pred) * 100
 accuracies[dim] = accuracy
 print(f" KNN Accuracy: {accuracy:.2f}%")

print("\n" + "="*70)
print("Distance Analysis Summary")
print("="*70)

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

dim_list = list(distance_stats.keys())
min_dists = [distance_stats[d]['min_mean'] for d in dim_list]
max_dists = [distance_stats[d]['max_mean'] for d in dim_list]
ratios = [distance_stats[d]['ratio_mean'] for d in dim_list]
```

```
axes[0, 0].plot(dim_list, min_dists, 'bo-', linewidth=2, markersize=8, label='Min Distance') axes[0, 0].plot(dim_list, max_dists,
'ro-', linewidth=2, markersize=8, label='Max Distance') axes[0, 0].set_xlabel('Dimensions', fontsize=12)
axes[0, 0].set_ylabel('Distance', fontsize=12)
axes[0, 0].set_title('Distance vs Dimensions', fontsize=14)
axes[0, 0].legend(fontsize=10)
axes[0, 0].grid(True, alpha=0.3)

axes[0, 1].plot(dim_list, ratios, 'go-', linewidth=2, markersize=8)
axes[0, 1].set_xlabel('Dimensions', fontsize=12)
axes[0, 1].set_ylabel('Ratio (Max/Min)', fontsize=12)
axes[0, 1].set_title('Distance Ratio vs Dimensions', fontsize=14)
axes[0, 1].grid(True, alpha=0.3)

axes[0, 2].plot(dim_list, list(accuracies.values()), 'mo-', linewidth=2, markersize=8) axes[0, 2].set_xlabel('Dimensions',
fontsize=12)
axes[0, 2].set_ylabel('Accuracy (%)', fontsize=12)
axes[0, 2].set_title('KNN Accuracy vs Dimensions', fontsize=14)
axes[0, 2].grid(True, alpha=0.3)
np.random.seed(42)
sample_data_2d = np.random.randn(100, 2)
distances_2d = squareform(pdist(sample_data_2d, metric='euclidean'))
axes[1, 0].hist(distances_2d[np.triu_indices(100, k=1)], bins=30, color='steelblue', edgecolor='black',  alpha=0.7)
axes[1, 0].set_xlabel('Distance', fontsize=12)
axes[1, 0].set_ylabel('Frequency', fontsize=12)
axes[1, 0].set_title('Distance Distribution (2D)', fontsize=14)
axes[1, 0].grid(True, alpha=0.3)

np.random.seed(42)
sample_data_100d = np.random.randn(100, 100)
distances_100d = squareform(pdist(sample_data_100d, metric='euclidean'))
axes[1, 1].hist(distances_100d[np.triu_indices(100, k=1)], bins=30, color='coral', edgecolor='black',  alpha=0.7)
axes[1, 1].set_xlabel('Distance', fontsize=12)
axes[1, 1].set_ylabel('Frequency', fontsize=12)
axes[1, 1].set_title('Distance Distribution (100D)', fontsize=14)
axes[1, 1].grid(True, alpha=0.3)

n_points = [10, 50, 100, 500, 1000]
dim_for_coverage = []
for n in n_points:
 dim = int(np.log(n) * 10)
 dim_for_coverage.append(dim)

axes[1, 2].plot(n_points, dim_for_coverage, 'ko-', linewidth=2, markersize=8)
axes[1, 2].set_xlabel('Number of Points', fontsize=12)
axes[1, 2].set_ylabel('Dimensions Needed', fontsize=12)
axes[1, 2].set_title('Points vs Dimensions for Coverage', fontsize=14)
axes[1, 2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('Q10_curse_of_dimensionality.png', dpi=150, bbox_inches='tight')
plt.close()

print("\n" + "="*70)
print("Key Observations")
print("="*70)
print("\n1. Distance Concentration:")
print(" - As dimensions increase, distances between points become more similar")
print(" - The ratio of max/min distance approaches 1 in high dimensions")
print(" - This makes distance-based algorithms less effective")

print("\n2. Model Performance:")
print(" - KNN accuracy generally decreases with increasing dimensions")
print(" - The 'curse' makes it harder to find meaningful neighbors")
print(" - More data is needed to maintain performance in high dimensions")

print("\n3. Data Sparsity:")
print(" - High-dimensional space is mostly empty")
print(" - Points become increasingly isolated")
print(" - Volume of space grows exponentially with dimensions")

print("\n4. Practical Implications:")
print(" - Feature selection/reduction is crucial")
print(" - Dimensionality reduction techniques (PCA, t-SNE) are important")
print(" - More training data needed for high-dimensional problems")

print("\nQ10 completed successfully!")
print("Plot saved as Q10_curse_of_dimensionality.png")
```

Dataset Used

Synthetic high-dimensional data generated using numpy.random.randn() for feature dimensions ranging from 2 to 100 — no external dataset required.

Output



Six-panel analysis: distance vs dimension, distance ratio, KNN accuracy vs dimension (dropping as dimensionality rises), distance-distribution histograms at 2D vs 100D (showing concentration around a narrow range at high dimension), and the growth of dimensions needed for coverage as sample size increases.